
BioVault Database Masterclass

From Absolute Zero to Confident Database Designer and Developer

BioVault Learning Edition

2026

Contents

Welcome: what this book will do for you	8
Who this course is for	8
What you will be able to do	8
How to use this book	8
The learning ladder	8
Part I — Absolute Zero: Data and Databases	10
1. Data: the smallest useful idea	10
1.1 What is data?	10
1.2 Data, information, and knowledge	10
1.3 Structured and unstructured data	10
1.4 Metadata	10
Beginner checkpoint	11
2. Why databases exist	11
2.1 The notebook problem	11
2.2 Database and DBMS are not the same thing	11
2.3 What a DBMS gives us	11
2.4 Relational databases	12
2.5 When a relational database is a good choice	12
Common mistake: “A spreadsheet is already a database”	12
3. Tables, rows, columns, and schemas	12
3.1 A table represents one type of thing or event	12
3.2 Rows are not defined by visible position	12
3.3 Data types	13
3.4 What does NULL mean?	13
3.5 Database schema: two meanings	13
Mini-lab	13
4. SQL: the language of relational databases	14
4.1 SQL is declarative	14
4.2 Major families of SQL commands	14
4.3 SQL keywords and style	14
4.4 Statements and semicolons	14
4.5 SQL dialects	14
Part II — From a Real Problem to a Data Model	16
5. Understand the biobank before designing tables	16
5.1 What is a biobank?	16
5.2 The BioVault problem statement	16
5.3 Scope	16
5.4 Stakeholders and needs	16
5.5 Requirements: functional and non-functional	17
Checkpoint	17
6. Business rules: turning reality into testable statements	17
6.1 What is a business rule?	17
6.2 BioVault’s major rules	17
6.3 Where should a rule be enforced?	18
6.4 Assumptions	18
6.5 Acceptance criteria	18
7. Entities and attributes	19
7.1 Entity	19
7.2 Entity type and entity instance	19
7.3 Attribute	19
7.4 Simple, composite, derived, and multivalued attributes	19
7.5 The BioVault entity inventory	19

8. Keys: how rows gain identity	20
8.1 Superkey, candidate key, and primary key	20
8.2 Natural and surrogate keys	20
8.3 Composite key	20
8.4 Foreign key	20
8.5 Referential actions	21
Common mistake: cascade everywhere	21
Key checkpoint	21
9. Relationships, cardinality, and participation	21
9.1 Relationship	21
9.2 Cardinality	21
9.3 Optionality and participation	22
9.4 Resolving many-to-many	22
9.5 Recursive relationship	22
9.6 Relationship attributes and event tables	22
Part III — ER Modeling, Relational Mapping, and Normalization	24
10. Reading and building the ER diagram	24
10.1 What an ER diagram shows	24
10.2 How to read the BioVault ERD	24
10.3 A repeatable ER-design method	24
10.4 Scenario validation	25
10.5 ERD versus physical schema	25
11. Map the ER model to relations	25
11.1 Strong entity mapping	25
11.2 One-to-many mapping	25
11.3 Many-to-many mapping	25
11.4 Optional relationships	26
11.5 Recursive mapping	26
11.6 Relational-schema notation	26
12. Normalization without fear	26
12.1 Why normalization exists	26
12.2 The three anomalies	27
12.3 Functional dependency	27
12.4 First Normal Form (1NF)	27
12.5 Second Normal Form (2NF)	27
12.6 Third Normal Form (3NF)	28
12.7 “The key, the whole key, and nothing but the key”	28
12.8 Why the audit log contains JSONB	28
12.9 Normalization lab	29
Part IV — Build the Database with PostgreSQL	30
13. PostgreSQL, tools, and project structure	30
13.1 Why PostgreSQL?	30
13.2 Client and server	30
13.3 Connection string	30
13.4 Why Docker is used	30
13.5 Why the SQL is split into files	31
13.6 The psql safety line	31
Setup checkpoint	31
14. DDL: create the structure	31
14.1 CREATE TABLE	31
14.2 Identity columns	32
14.3 Choosing text types	32
14.4 Exact numeric data	32
14.5 DATE and TIMESTAMPTZ	32
14.6 Defaults	33

14.7 Rebuild strategy	33
Common DDL mistakes	33
15. Constraints: make invalid states difficult	33
15.1 NOT NULL	33
15.2 UNIQUE	33
15.3 CHECK	33
15.4 Checks and NULL	34
15.5 Foreign keys	34
15.6 Constraint naming	34
15.7 Constraint versus trigger	34
15.8 Negative testing	34
16. DML: insert, update, delete	35
16.1 INSERT	35
16.2 Load parents before children	35
16.3 UPDATE	35
16.4 DELETE	36
16.5 A reversible demonstration	36
16.6 Idempotent and deterministic scripts	36
17. Seed data that teaches and tests	36
17.1 Test data is part of the design	36
17.2 Main-table row counts	36
17.3 Synthetic does not mean random nonsense	37
17.4 Verify counts	37
Part V — Query SQL from First SELECT to Advanced Analytics	38
18. Basic SELECT	38
18.1 The query pipeline	38
18.2 Select specific columns	38
18.3 Aliases	38
18.4 Filtering with WHERE	38
18.5 Pattern matching	39
18.6 DISTINCT	39
18.7 CASE	39
Basic query lab	39
19. Joins: reconnect normalized facts	39
19.1 Why joins are necessary	39
19.2 Inner join	40
19.3 Multi-table lineage join	40
19.4 Left join	40
19.5 A subtle outer-join mistake	40
19.6 Self join	40
19.7 Join multiplication	41
Join lab	41
20. Aggregation, grouping, and HAVING	41
20.1 Aggregate functions	41
20.2 GROUP BY	41
20.3 COALESCE	42
20.4 WHERE versus HAVING	42
20.5 Filtered aggregates	42
Aggregation lab	42
21. Subqueries and EXISTS	42
21.1 Scalar subquery	42
21.2 Correlated subquery	43
21.3 IN and EXISTS	43
21.4 Two independent existence rules	43
21.5 NOT IN and null danger	44

22. Common Table Expressions and recursion	44
22.1 CTE	44
22.2 Recursive CTE structure	44
22.3 Think through one iteration	45
22.4 Cycle protection	45
23. Window functions	45
23.1 Aggregate versus window	45
23.2 The window definition	45
23.3 Ranking functions	46
23.4 Running total example	46
23.5 Previous value with LAG	46
Part VI — Reusable SQL, Performance, and Transactional Correctness	47
24. Views: saved relational questions	47
24.1 What is a view?	47
24.2 Why use a view?	47
24.3 View versus table	47
24.4 View design	47
24.5 Security note	48
25. Indexes and query planning	48
25.1 The book-index analogy	48
25.2 The cost of an index	48
25.3 Constraint-created indexes	48
25.4 Composite index order	48
25.5 Partial index	48
25.6 EXPLAIN	49
25.7 Sequential scan is not automatically bad	49
25.8 Sargability	49
Performance checkpoint	49
26. Transactions and ACID	49
26.1 What is a transaction?	49
26.2 ACID	49
26.3 COMMIT and ROLLBACK	50
26.4 Savepoints	50
26.5 Keep transactions focused	50
27. Concurrency and row locks	50
27.1 The lost-stock race	50
27.2 Row-level locking	50
27.3 Why “check then update” must share a transaction	51
27.4 Isolation levels	51
27.5 Deadlocks	51
28. Functions, procedures, and triggers	51
28.1 Function	51
28.2 Procedure	52
28.3 Trigger	52
28.4 Why use a public function plus a trigger?	52
28.5 Trigger workflow	52
28.6 Detailed business-rule logic	53
28.7 Trigger risks	53
29. Auditing with OLD, NEW, and JSONB	53
29.1 Audit objective	53
29.2 Generic audit row	54
29.3 Why AFTER?	54
29.4 CURRENT_USER	54
29.5 Audit is not backup	54

30. PostgreSQL views and logic: a guided project reading	54
Part VII — Build the Bonus CRUD Application	56
31. CRUD and application architecture	56
31.1 What CRUD means	56
31.2 BioVault application layers	56
31.3 Why Tkinter?	56
31.4 PostgreSQL and SQLite modes	57
32. Connections and transaction handling in Python	57
32.1 Context manager	57
32.2 Environment configuration	58
32.3 Driver	58
32.4 Foreign keys in SQLite	58
32.5 Connection scope	58
33. Safe SQL from application code	58
33.1 Parameterized queries	58
33.2 Why placeholders differ	59
33.3 Dynamic identifiers are different	59
33.4 Search pattern	59
33.5 Return only required fields	59
34. Validation and error handling	59
34.1 Validation layers	59
34.2 Cleaning first	60
34.3 Regex validation	60
34.4 Enumerated application sets	60
34.5 Dates	60
34.6 Do not hide exceptions	60
35. Repository pattern and real CRUD	60
35.1 Repository responsibility	60
35.2 Create and return the identifier	61
35.3 Detect missing updates/deletes	61
35.4 Protected deletion	61
35.5 Read queries are also real database features	61
Part VIII — Testing, Evidence, Delivery, and Defense	62
36. Testing databases and applications	62
36.1 Testing pyramid for this project	62
36.2 Positive and negative tests	62
36.3 Arrange, Act, Assert	62
36.4 Isolated test database	62
36.5 Parameterized test	62
36.6 Test commands	63
36.7 What row counts do and do not prove	63
36.8 Test evidence	63
37. Debugging method	64
37.1 Read the first real error	64
37.2 Reduce the problem	64
37.3 Use transactions for experiments	64
37.4 Inspect schema, do not guess	64
37.5 Common error categories	64
38. Git and repository organization	65
38.1 Why version control matters	65
38.2 Useful workflow	65
38.3 Repository structure communicates quality	65
38.4 Generated and source artifacts	65

39. Report, presentation, demo, and oral defense	65
39.1 Tell one coherent story	65
39.2 Defend decisions, not syntax alone	66
39.3 Live demonstration sequence	66
39.4 Likely defense questions	66
39.5 Be honest about limitations	67
Part IX — Guided Labs	68
40. Lab 1: create a tiny clean schema	68
Goal	68
41. Lab 2: normalize a laboratory spreadsheet	68
42. Lab 3: query the BioVault lineage	68
43. Lab 4: analytical SQL	69
44. Lab 5: transaction and concurrency reasoning	69
45. Lab 6: add one application feature safely	69
46. Lab 7: design acceptance tests	69
Part X — Answer Key and Explanations	71
47. Checkpoint answers	71
47.1 Chapter 1	71
47.2 Chapter 3	71
47.3 Chapter 5	71
47.4 Chapter 8	71
47.5 Chapter 13	71
47.6 Chapter 25	71
48. Lab solution sketches	71
48.1 Lab 1	71
48.2 Lab 2	72
48.3 Lab 3	72
48.4 Lab 4 selected solutions	73
48.5 Lab 5	73
48.6 Lab 6 design	73
48.7 Lab 7 example	74
Part XI — Cheat Sheets	75
49. SQL statement cheat sheet	75
Create	75
Insert	75
Read	75
Update	75
Delete	75
Aggregate	75
Transaction	75
CTE	76
Window	76
50. Design review cheat sheet	76
51. PostgreSQL command cheat sheet	76
Part XII — Glossary	78
52. Database glossary	78

53. Final learning path: from student to expert	80
Stage 1 — Explain	80
Stage 2 — Model	80
Stage 3 — Normalize	80
Stage 4 — Build and query	80
Stage 5 — Protect	80
Stage 6 — Integrate	80
Stage 7 — Defend	80
54. Final project self-assessment	81
55. References and further study	82
Repository sources to study beside this book	82
Closing note	82
56. Table-by-table BioVault deep dive	83
56.1 donors	83
56.2 consent_types	83
56.3 consents	83
56.4 researchers	83
56.5 research_projects	83
56.6 project_researchers	84
56.7 sample_types	84
56.8 collection_events	84
56.9 samples	84
56.10 storage_units	84
56.11 aliquots	85
56.12 test_types	85
56.13 test_requests	85
56.14 sample_usage	85
56.15 audit_log	85
56.16 Trace one question across the schema	85
57. Beginner mistakes and expert corrections	86
57.1 Designing from screens	86
57.2 Adding IDs but ignoring candidate keys	86
57.3 Treating null, zero, and blank as identical	86
57.4 Storing lists in one column	86
57.5 Copying descriptive data into transaction tables	86
57.6 Using a join without knowing grain	86
57.7 Confusing WHERE and HAVING	86
57.8 Hiding duplicates with DISTINCT	86
57.9 Building SQL with user strings	87
57.10 Checking stock only in the UI	87
57.11 Catching every error and continuing	87
57.12 Optimizing before measuring	87
57.13 Testing only success	87
57.14 Memorizing the presentation	87

Welcome: what this book will do for you

This book teaches databases through one complete project: **BioVault**, a biobank and biospecimen management system. You do not need previous database, programming, or biotechnology knowledge. We start with the meaning of a piece of data and finish with transactions, row locks, triggers, window functions, recursive queries, a connected Python application, automated tests, and project defense.

The course is intentionally progressive. Early chapters use ordinary language and tiny examples. Later chapters use professional terminology and the real BioVault implementation. Whenever a difficult word appears, it is defined before it is used.

The central learning idea: do not memorize SQL as a collection of magic commands. First understand the real-world rule, then the data model, and only then the SQL that enforces or answers it.

Who this course is for

This course is suitable if any of the following sounds like you:

- You have never created a database.
- You can write a `SELECT`, but joins or normalization still feel confusing.
- You built the project and want to understand every design choice before the oral defense.
- You want to move from classroom SQL to safe, testable application code.
- You know basic databases and want the advanced reasoning behind this project.

What you will be able to do

By the end, you should be able to:

1. explain what a relational database is without using unexplained jargon;
2. turn a real problem into requirements and business rules;
3. identify entities, attributes, keys, cardinality, and participation;
4. draw and defend an ER diagram;
5. transform a conceptual model into relations in Third Normal Form;
6. create PostgreSQL tables, constraints, indexes, views, functions, and triggers;
7. write basic, intermediate, and advanced SQL queries;
8. explain transactions, ACID, concurrency, and row locking;
9. connect a Python CRUD interface safely to PostgreSQL;
10. write positive and negative automated tests;
11. organize, demonstrate, and defend a complete database project.

How to use this book

Each chapter contains four kinds of material:

- **Plain-English idea:** the concept without technical decoration.
- **BioVault example:** where the concept appears in the real project.
- **Common mistake:** what beginners often do and why it causes trouble.
- **Checkpoint or lab:** a small task that proves you understood the idea.

Read Parts I–III in order if you are new. If you already understand relational design, begin with Part IV but return to normalization before studying triggers. The answer key is near the end, but attempt the questions before looking.

The learning ladder

Level	Focus	You are ready to advance when...
Level 0 — Absolute beginner	data, tables, rows, columns	you can describe a table accurately
Level 1 — Modeler	rules, entities, keys, relationships	you can explain the ERD
Level 2 — SQL builder	DDL, DML, constraints, joins	you can build and query the schema
Level 3 — Analyst	aggregates, subqueries, CTEs, windows	you can answer non-trivial questions

Level	Focus	You are ready to advance when...
Level 4 — Database engineer	transactions, triggers, locks, indexes	you can protect correctness under failure
Level 5 — Application developer	CRUD, validation, testing	you can connect a safe interface
Level 6 — Project defender	trade-offs, evidence, live demo	you can justify and modify the work

Part I — Absolute Zero: Data and Databases

1. Data: the smallest useful idea

1.1 What is data?

Data is a recorded fact or observation. Examples are:

- a donor code: BIO-D0001;
- a collection date: 2026-01-15;
- a quantity: 0.750 mL;
- a status: AVAILABLE;
- a yes/no value: true.

By itself, a value may be ambiguous. The value 12 could mean 12 donors, 12 milliliters, or the twelfth freezer. Data becomes useful when we add **context**: a label, a unit, a time, and a relationship to other facts.

Compare:

0.750

with:

Aliquot ALQ-00001 currently contains 0.750 mL.

The second statement can support a decision. A scientist can decide whether enough material is available for a test.

1.2 Data, information, and knowledge

These words are related but not identical:

- **Data**: stored facts, such as individual usage quantities.
- **Information**: data organized to answer a question, such as total quantity used by each project.
- **Knowledge**: an interpretation used for action, such as deciding to reject a new request because stock is too low.

A database stores data. Queries transform data into information. A person or business process uses that information as knowledge.

1.3 Structured and unstructured data

Structured data follows a known shape. Every donor record has a donor code, status, and registration date. This shape works naturally in tables.

Unstructured data does not follow one strict row-and-column form. Images, free-text reports, audio, and raw sequencing files are examples.

BioVault stores structured facts and some short text notes. It does not store large raw genomic files; a production system would usually store those files in specialized object storage and keep their identifiers and metadata in the database.

1.4 Metadata

Metadata is data that describes other data. For a sample, metadata can include:

- specimen type;
- collection time;
- processing method;
- storage temperature;
- quality status.

The physical specimen is not inside PostgreSQL. The database stores the metadata needed to find, understand, and govern it.

Beginner checkpoint

Classify each item:

1. BIO-D0007
2. a microscope image
3. the statement “this image was captured at 40× magnification”
4. a report counting available plasma aliquots

The answers are in the final answer key.

2. Why databases exist

2.1 The notebook problem

Imagine a laboratory recording specimens in a paper notebook. One person can write at a time. Searching is slow. Handwriting may be unclear. A page may be lost. Two people might assign the same sample code.

A spreadsheet improves searching and calculation, but large connected processes create new problems:

- the same donor details are copied into many rows;
- correcting one fact requires many edits;
- two users can overwrite one another;
- there is no strong guarantee that a referenced donor exists;
- deleting a row may accidentally remove the only copy of an important fact;
- permission and audit rules are difficult to enforce consistently.

A database management system is built to solve these problems.

2.2 Database and DBMS are not the same thing

A **database** is an organized collection of data.

A **Database Management System (DBMS)** is the software that stores, retrieves, protects, and coordinates that data. PostgreSQL, MySQL, SQL Server, Oracle Database, and SQLite are DBMS products.

An analogy:

Analogy	Database world
books and records	data
organized library collection	database
librarian and library rules	DBMS
request slip	query

BioVault uses **PostgreSQL** as its official DBMS. It also includes a small SQLite demonstration database so the desktop interface can start immediately. SQLite is a convenience here, not the graded source of truth.

2.3 What a DBMS gives us

A serious DBMS provides:

- persistent storage;
- a query language;
- data type checking;
- integrity constraints;
- concurrent access;
- transactions and recovery;
- authentication and permissions;
- indexes for performance;
- backup and restore tools;
- logs and monitoring.

2.4 Relational databases

A **relational database** represents data as relations. In practical SQL work, we normally see relations as tables.

The relational model is powerful because:

1. each table has a clear meaning;
2. rows follow one defined structure;
3. keys identify rows;
4. foreign keys connect related rows;
5. operations can combine tables without copying all facts together.

The word *relational* comes from the mathematical idea of a relation, not merely from the fact that tables have relationships.

2.5 When a relational database is a good choice

Use a relational database when:

- the data has consistent structure;
- correctness rules matter;
- records reference one another;
- transactions must keep several changes together;
- flexible reporting is important.

BioVault fits all five conditions. An aliquot use must create a history record and deduct inventory together. If either action fails, neither should remain.

Common mistake: “A spreadsheet is already a database”

A spreadsheet can hold tabular data and may be completely adequate for a small, single-user task. It does not normally provide the same declarative constraints, transaction isolation, relationship enforcement, or concurrent-write control as a DBMS. The right question is not “Can a spreadsheet store these cells?” but “Can it reliably enforce the complete process?”

3. Tables, rows, columns, and schemas

3.1 A table represents one type of thing or event

Consider a tiny donor table:

donor_id	donor_code	blood_type	donor_status
1	BIO-D0001	A+	ACTIVE
2	BIO-D0002	O-	ACTIVE

The table is named donors.

- A **column** describes one property, such as `blood_type`.
- A **row** describes one occurrence, such as donor 1.
- A **cell** is one column value in one row.
- The **table definition** specifies what values are allowed.

A good table has one clear subject. `donors` stores donor facts. It should not also contain an unlimited list of samples or projects in one cell.

3.2 Rows are not defined by visible position

In a spreadsheet, people say “row 17.” In a relational table, row order is not guaranteed unless a query uses `ORDER BY`. A row is identified by its key, not by where it happens to appear on screen.

This query gives a defined order:

```
SELECT donor_code, donor_status
FROM donors
ORDER BY donor_code;
```

Without ORDER BY, PostgreSQL may return the same rows in a different order after an index, update, or execution-plan change.

3.3 Data types

A **data type** defines the kind of value a column accepts and which operations make sense.

Type	Example use	Why it matters
VARCHAR(20)	donor code	limited text
TEXT	notes	variable-length text
SMALLINT	birth year	whole number
NUMERIC(14,3)	quantity	exact decimal value
DATE	consent grant date	calendar date
TIMESTAMPTZ	collection instant	instant with time-zone awareness
BOOLEAN	active flag	true or false
JSONB	audit snapshot	structured JSON document

Do not store every value as text. If quantity is text, the database cannot reliably compare, sum, or validate it. If a date is text, 31/12/2026, 12/31/2026, and 2026-12-31 may be confused.

3.4 What does NULL mean?

NULL means **missing, unknown, or not applicable**. It is not zero, an empty string, or the word "NULL".

For example, expires_on IS NULL means the consent has no recorded expiry date. A blood_type can be null when it was not collected.

SQL uses three-valued logic: a comparison can be true, false, or unknown. Therefore this is wrong:

```
WHERE blood_type = NULL
```

Use:

```
WHERE blood_type IS NULL
```

and:

```
WHERE blood_type IS NOT NULL
```

3.5 Database schema: two meanings

The word **schema** is used in two related ways:

1. the logical structure of tables, columns, keys, and constraints;
2. a PostgreSQL namespace that groups objects.

BioVault creates a PostgreSQL namespace named biobank:

```
CREATE SCHEMA biobank;
SET search_path TO biobank, public;
```

The full name biobank.donors removes ambiguity. The search_path allows SQL inside the project scripts to use the shorter name donors.

Mini-lab

For each value, choose a sensible data type:

1. an aliquot quantity with three decimal places;
2. whether a researcher is active;
3. the exact instant a sample arrived;

4. a long explanation of a restriction;
5. the date on which a project begins.

4. SQL: the language of relational databases

4.1 SQL is declarative

SQL stands for **Structured Query Language**. Most SQL is declarative: you describe the result or rule you want, and the DBMS chooses an execution method.

```
SELECT donor_code
FROM donors
WHERE donor_status = 'ACTIVE';
```

This says what rows are wanted. It does not tell PostgreSQL whether to scan a table, use an index, or combine several internal techniques.

4.2 Major families of SQL commands

Family	Purpose	Examples
DDL	define structure	CREATE, ALTER, DROP
DML	insert or change rows	INSERT, UPDATE, DELETE
DQL	retrieve rows	SELECT
TCL	control transactions	BEGIN, COMMIT, ROLLBACK
DCL	control privileges	GRANT, REVOKE

Some books classify SELECT as part of DML. The classification is less important than understanding the effect of each statement.

4.3 SQL keywords and style

SQL keywords are usually case-insensitive, but this course writes keywords in uppercase and object names in snake_case:

```
SELECT sample_code, initial_quantity
FROM samples
WHERE quality_status = 'ACCEPTED'
ORDER BY sample_code;
```

This formatting is a reading aid, not a database requirement.

4.4 Statements and semicolons

A semicolon ends an SQL statement:

```
SELECT COUNT(*) FROM donors;
SELECT COUNT(*) FROM samples;
```

In interactive tools, forgetting the semicolon often makes the prompt wait because it assumes the statement is incomplete.

4.5 SQL dialects

SQL is standardized, but DBMS products add features and syntax. BioVault uses PostgreSQL features including:

- identity columns;
- TIMESTAMPTZ;
- ILIKE;
- JSONB;
- FILTER;
- PL/pgSQL functions and triggers;

- partial indexes;
- FOR UPDATE.

That is why the main scripts target PostgreSQL 16 or later. The SQLite demo contains a simplified compatible subset for the interface.

Part II — From a Real Problem to a Data Model

5. Understand the biobank before designing tables

5.1 What is a biobank?

A biobank is an organized collection of biological specimens and associated data used for approved research. Specimens can include blood, plasma, serum, tissue, DNA, RNA, cells, and other materials.

The database does not replace laboratory science. It provides traceability:

Donor

- > Consent
- > Collection event
- > Parent sample
- > Aliquot
- > Storage position
- > Test or authorized research use

An **aliquot** is a smaller portion prepared from a sample. Dividing a sample into aliquots reduces repeated handling of the parent material and lets different studies consume controlled quantities.

5.2 The BioVault problem statement

BioVault must manage pseudonymous human research specimens from registration through consent, collection, processing, aliquoting, storage, testing, and authorized consumption.

It must answer questions such as:

- What usable material exists right now?
- Where exactly is each aliquot?
- From which donor and collection event did a sample originate?
- Was research consent valid on the usage date?
- Was the researcher assigned to the project?
- Did the use exceed remaining stock?
- What critical data changed, when, and by whom?

5.3 Scope

In scope:

- pseudonymous donor records;
- versioned consent;
- researchers and ethics-approved projects;
- collection events, samples, and aliquots;
- hierarchical storage;
- test requests and results;
- controlled research usage;
- audit history;
- operational views;
- a connected CRUD interface.

Out of scope:

- names, national identifiers, addresses, and phone numbers;
- clinical diagnosis or treatment;
- billing and insurance;
- instrument control;
- raw sequencing files;
- the external identity-linking key.

Scope protects a project from becoming endless. A feature can be useful and still be outside this project.

5.4 Stakeholders and needs

Stakeholder	Questions or actions
Biobank manager	What is available? Is consent valid?
Laboratory scientist	Where is the aliquot? What tests are pending?
Researcher	Which samples may my project use?
Quality specialist	Which samples are quarantined or rejected?
Database administrator	Can we deploy, secure, back up, and recover it?
Auditor or instructor	Are rules enforced and results reproducible?

5.5 Requirements: functional and non-functional

A **functional requirement** describes what the system does:

- create and update donor records;
- record collection events;
- find available samples;
- deduct consumed quantity.

A **non-functional requirement** describes a quality or constraint:

- invalid quantities must be rejected;
- operations must be auditable;
- two simultaneous uses must not overdraw stock;
- the project must rebuild from scripts;
- the interface must respond with clear errors.

Both matter. “Record usage” is incomplete without “record usage safely.”

Checkpoint

Explain why “store patient names” is not merely an extra column in this project. Consider scope, privacy risk, access control, and whether the research database needs the value.

6. Business rules: turning reality into testable statements

6.1 What is a business rule?

A **business rule** is a precise statement that defines or constrains the operation of the organization. It is not necessarily about money; “business” means the activity being modeled.

A weak statement:

Quantities should be sensible.

A testable rule:

An aliquot’s initial quantity must be greater than zero, and current quantity must be between zero and initial quantity.

The second statement can become a database constraint and a test.

6.2 BioVault’s major rules

1. Every donor has one unique code matching BIO-D0000.
2. A donor may grant several consent types and versions.
3. Consent expiry cannot precede the grant date.
4. Every collection event belongs to exactly one donor and one collector.
5. Every sample comes from exactly one collection event and one sample type.
6. Initial quantities are positive.
7. Remaining aliquot quantity cannot be negative or exceed initial quantity.
8. One storage position holds at most one aliquot.
9. Storage units form a hierarchy.
10. Researchers and projects have a many-to-many relationship.
11. Research use requires active RESEARCH_USE consent on the use date.
12. The recording researcher must be an assigned project member on that date.

13. The project must be authorized on that date.
14. A use must lock and deduct stock atomically.
15. Important updates and deletes must be audited.
16. A completed test requires a valid completion date.
17. Scientific history cannot be casually deleted.

6.3 Where should a rule be enforced?

Rules can be enforced at several layers:

Layer	Example	Strength
User interface	dropdown for donor status	friendly feedback
Application	regex validates donor code	reusable application logic
Database constraint	CHECK validates donor code	protects every client
Trigger/function	validates consent and inventory together	protects complex writes
Human process	ethics committee approval	handles judgment outside software

Important structural rules belong in the database even if the interface also checks them. Otherwise a script, import tool, or second application can bypass the rule.

Use the simplest mechanism that correctly expresses the rule:

- NOT NULL for required values;
- UNIQUE for candidate keys;
- CHECK for rules within one row;
- FOREIGN KEY for relationships;
- a transaction or trigger for rules that span several rows and actions.

6.4 Assumptions

An assumption fills a gap so the design can proceed. It must be written down. BioVault assumes:

- codes are pseudonymous;
- all submitted records are synthetic;
- timestamps use TIMESTAMPTZ;
- quantities are not summed across incompatible units;
- each physical aliquot occupies one unique box position;
- PostgreSQL is the official target;
- SQLite exists only for portable UI demonstration.

An assumption is not permission to ignore uncertainty. It is a visible design decision that stakeholders can confirm or change.

6.5 Acceptance criteria

Turn each critical rule into evidence:

Rule	Acceptance test
donor code format	insert malformed code and expect rejection
remaining quantity	insert current quantity greater than initial and expect rejection
valid use	insert authorized usage and verify deduction
no overdraw	request more than available and expect rejection
active consent	use expired-consent material and expect rejection
membership	use by unassigned researcher and expect rejection
auditing	update a donor and verify an audit row

This is the beginning of test-driven database thinking: decide how correctness will be proved before claiming the design is correct.

7. Entities and attributes

7.1 Entity

An **entity** is a distinguishable real-world thing, concept, or event that the system needs to remember.

BioVault entities include:

- donor — a person represented pseudonymously;
- sample — collected biological material;
- project — approved research work;
- collection event — something that happened at a time and place;
- sample usage — a consumption event.

Entities are not limited to physical objects. Events and associations can also deserve their own identity and attributes.

7.2 Entity type and entity instance

Donor is an entity type: the general category.

BIO-D0001 is an entity instance: one particular donor.

In implementation, an entity type often becomes a table and an entity instance often becomes a row.

7.3 Attribute

An **attribute** is a property that describes an entity:

- donor has donor_code, birth_year, and donor_status;
- sample has sample_code, initial_quantity, and quality_status;
- usage has used_on, quantity_used, and purpose.

Ask three questions for every proposed attribute:

1. What exactly does it mean?
2. Can it contain one atomic value?
3. Does it describe this entity, or a different entity?

researcher_email does not describe a usage event; it describes a researcher. Store the researcher reference on the usage and keep the email in researchers.

7.4 Simple, composite, derived, and multivalued attributes

- **Simple:** cannot usefully be divided for the system, such as status.
- **Composite:** has meaningful components, such as a postal address. Direct addresses are outside BioVault.
- **Derived:** calculated from other facts, such as remaining percentage.
- **Multivalued:** can have several values, such as a donor's consent records.

Relational design normally represents a repeating multivalued fact in a child table, not in comma-separated text.

Bad:

```
donors.consent_types = "research,genetics,commercial"
```

Better:

```
one donors row
many consents rows
each consent references one consent_types row
```

7.5 The BioVault entity inventory

Area	Entities
Donor governance	donors, consent_types, consents

Area	Entities
People and projects	researchers, research_projects, project_researchers
Specimen lineage	collection_events, sample_types, samples, aliquots
Physical inventory	storage_units
Laboratory work	test_types, test_requests
Controlled use	sample_usage
Technical history	audit_log

There are 15 tables. The bridge `project_researchers` is a full relation, not a hidden arrow, because project membership has its own role and dates.

8. Keys: how rows gain identity

8.1 Superkey, candidate key, and primary key

A **superkey** is any set of columns that uniquely identifies a row.

A **candidate key** is a minimal superkey: remove any column and it is no longer guaranteed unique.

The **primary key** is the candidate key chosen as the main row identifier.

For donors:

- `donor_id` is the primary key;
- `donor_code` is another candidate key protected by `UNIQUE`.

8.2 Natural and surrogate keys

A **natural key** has business meaning, such as `BIO-D0001`.

A **surrogate key** is introduced by the system, such as numeric `donor_id`.

BioVault uses both:

```
donor_id    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
donor_code  VARCHAR(20) NOT NULL,
CONSTRAINT uq_donors_code UNIQUE (donor_code)
```

Why?

- the numeric key is compact and stable for joins;
- the code is readable to users and remains uniquely enforced;
- if the external code format changes, foreign keys do not all need to change.

A surrogate key does **not** remove the need to declare real candidate keys. Without `UNIQUE (donor_code)`, duplicate business identities would be possible.

8.3 Composite key

A **composite key** contains more than one column.

```
PRIMARY KEY (project_id, researcher_id)
```

in `project_researchers` says the same researcher cannot be entered twice for the same project. Neither column alone is unique because:

- one project has many researchers;
- one researcher joins many projects.

8.4 Foreign key

A **foreign key** requires a value to match a candidate key in another table.

```
FOREIGN KEY (donor_id)
REFERENCES donors(donor_id)
```

This prevents an orphan collection event that points to a nonexistent donor. The referenced row is the **parent**; the referencing row is the **child**. These words describe the relationship, not importance.

8.5 Referential actions

What happens if a parent row is deleted?

- **RESTRICT/NO ACTION**: reject deletion while children exist.
- **CASCADE**: delete matching children.
- **SET NULL**: keep children but remove the reference.

BioVault mostly uses restrictive deletion because scientific traceability should not disappear. It uses cascade for project membership because a membership has no independent meaning after its project is removed.

Common mistake: cascade everywhere

Cascade is convenient, but a single donor deletion could otherwise erase events, samples, aliquots, tests, and usage history. Convenience is not a valid reason to destroy evidence.

Key checkpoint

For each column, classify it:

1. `samples.sample_id`
2. `samples.sample_code`
3. `samples.collection_event_id`
4. `(project_id, researcher_id)` in `project_researchers`

9. Relationships, cardinality, and participation

9.1 Relationship

A **relationship** describes how entity instances are associated.

Examples:

- a donor **has** collection events;
- a collection event **produces** samples;
- a sample **is divided into** aliquots;
- a project **includes** researchers.

Relationship names should read like meaningful sentences in both directions.

9.2 Cardinality

Cardinality states the maximum number of instances that can participate.

- one-to-one (1:1);
- one-to-many (1:M);
- many-to-many (M:N).

BioVault examples:

Relationship	Cardinality
donor → collection events	1:M
collection event → samples	1:M
sample → aliquots	1:M
project ↔ researcher	M:N

9.3 Optionality and participation

Participation states whether involvement is required.

- **Total participation:** a child cannot exist without the relationship.
- **Partial participation:** an entity may exist without a matching row.

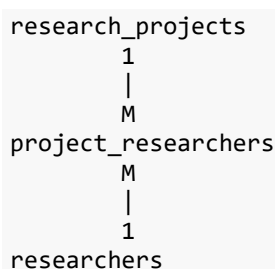
Every sample must reference one collection event, so sample participation is total. A newly registered donor may not have a collection event yet, so donor participation in that relationship is partial.

Crow's-foot notation often expresses minimum and maximum together:

- 0..1 — zero or one;
- 1..1 — exactly one;
- 0..many — optional many;
- 1..many — at least one.

9.4 Resolving many-to-many

Relational tables do not implement a direct M:N relationship with a single foreign key. Create an associative table:



The bridge also stores facts about the association:

- project_role;
- joined_on;
- left_on.

These values belong neither to the project alone nor to the researcher alone. They describe one researcher's membership in one project.

9.5 Recursive relationship

storage_units references itself:

```

parent_storage_unit_id BIGINT,
FOREIGN KEY (parent_storage_unit_id)
REFERENCES storage_units(storage_unit_id)

```

This models:

```

Facility
-> Room
  -> Freezer
    -> Rack
      -> Box

```

The root facility has a null parent. The same table supports every level, and a recursive CTE can reconstruct the path.

9.6 Relationship attributes and event tables

Sometimes an association becomes important enough to have its own identity. sample_usage connects an aliquot, project, and researcher, but also records:

- date;
- quantity;
- purpose;
- recording timestamp.

It is not merely a bridge. It is an auditable business event.

Part III — ER Modeling, Relational Mapping, and Normalization

10. Reading and building the ER diagram

10.1 What an ER diagram shows

An **Entity–Relationship Diagram (ERD)** is a conceptual map of:

- entity types;
- important attributes;
- identifiers;
- relationships;
- cardinality and optionality.

It helps people review the model before implementation details become expensive.

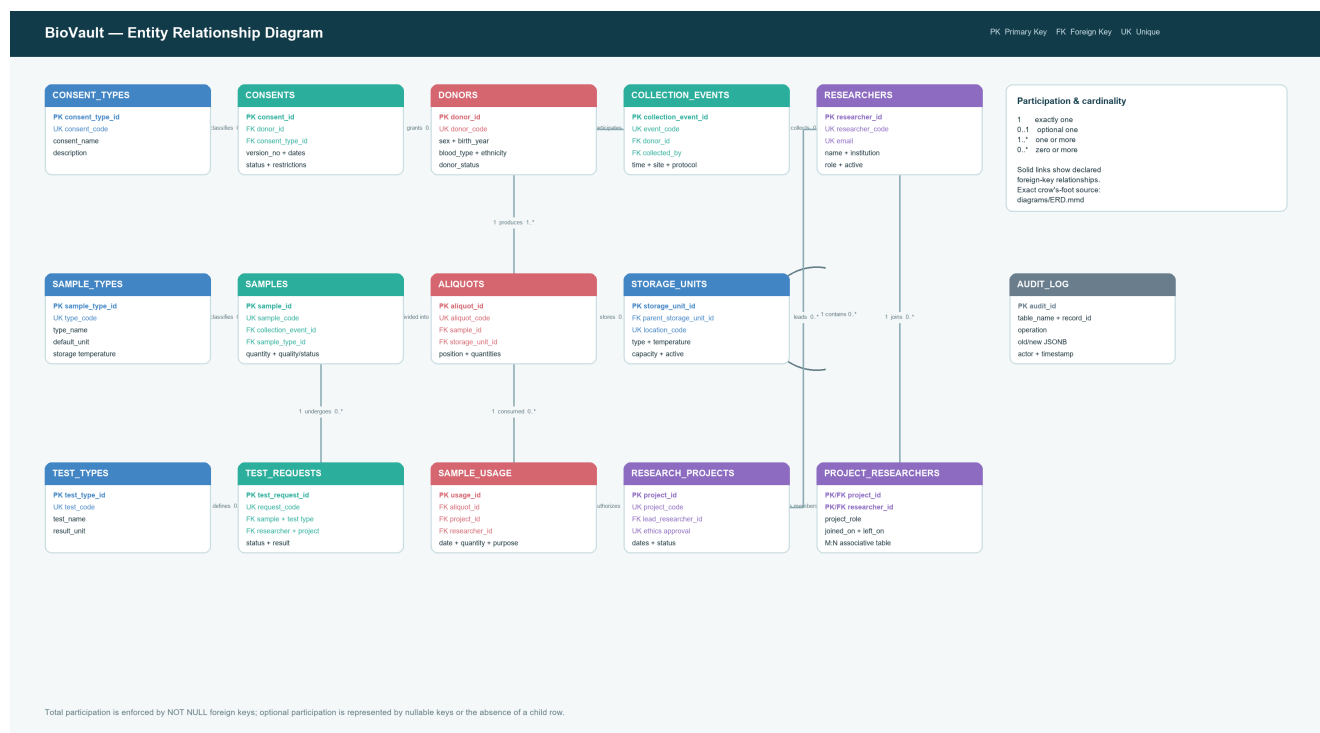


Figure 1: BioVault ER diagram. The editable Mermaid source is in `diagrams/ERD.mmd`.

10.2 How to read the BioVault ERD

Start from the center of the scientific lineage:

1. DONORS have COLLECTION_EVENTS.
2. An event produces SAMPLES.
3. A sample is divided into ALIQUOTS.
4. An aliquot is placed in a STORAGE_UNIT.
5. A SAMPLE_USAGE consumes some aliquot quantity.
6. The usage is authorized by a RESEARCH_PROJECT and recorded by a RESEARCHER.

Then read the governance branches:

- CONSENTS connect donors to controlled CONSENT_TYPES.
- PROJECT_RESEARCHERS records who belongs to a project.
- TEST_REQUESTS connect samples, test types, researchers, and optionally projects.

10.3 A repeatable ER-design method

1. Write the system purpose in one paragraph.

2. List stakeholder questions.
3. Extract nouns as candidate entities.
4. Extract properties as candidate attributes.
5. Extract verbs as candidate relationships.
6. identify keys.
7. state minimum and maximum cardinalities.
8. resolve every M:N relationship.
9. test the model with real scenarios.
10. remove duplication and attributes that belong elsewhere.

Do not blindly turn every noun into a table. “Report” might be a query result, not a stored entity. Do not blindly turn every verb into a table either. “Sample has a type” needs a foreign key; “sample is consumed” needs an event table because date and quantity matter.

10.4 Scenario validation

Walk a scenario through the diagram:

Donor BIO-D0001 grants research consent. A scientist records a blood collection. The laboratory creates a plasma sample, divides it into two aliquots, stores them in different positions, requests a test, and later consumes 0.050 mL for project PRJ-001.

If the diagram cannot represent a required scenario without stuffing multiple facts into one attribute, the model is incomplete.

10.5 ERD versus physical schema

An ERD communicates meaning. A physical schema adds DBMS details:

- exact types;
- constraint names;
- indexes;
- defaults;
- generated identity behavior;
- referential actions.

The ERD and DDL should agree, but they serve different audiences.

11. Map the ER model to relations

11.1 Strong entity mapping

Each regular entity becomes a table. Its identifier becomes a primary key.

```
DONOR(donor_id, donor_code, sex_at_birth, ...)
```

11.2 One-to-many mapping

Place the foreign key on the many side.

```
DONORS 1 ---- M COLLECTION_EVENTS
```

becomes:

```
COLLECTION_EVENTS(..., donor_id FK -> DONORS)
```

Putting event_id on the donor row would allow only one event unless you added repeating columns such as event_1, event_2, which is not relational.

11.3 Many-to-many mapping

Create a bridge relation containing the two parent keys:

```
PROJECT_RESEARCHERS(
  project_id PK/FK,
  researcher_id PK/FK,
  project_role,
  joined_on,
  left_on
)
```

11.4 Optional relationships

`test_requests.project_id` is nullable because a quality-control test may not belong to a research project. The relationship is optional for the request.

Use nullability only when “no related row” is meaningful. Do not make every foreign key nullable merely to make inserts easier.

11.5 Recursive mapping

A hierarchy with the same kind of object at each level uses a self-foreign key:

```
STORAGE_UNITS(
  storage_unit_id PK,
  parent_storage_unit_id FK -> STORAGE_UNITS,
  ...
)
```

11.6 Relational-schema notation

This compact notation is common in reports:

```
SAMPLES(
  sample_id PK,
  sample_code UQ NN,
  collection_event_id FK -> COLLECTION_EVENTS NN,
  sample_type_id FK -> SAMPLE_TYPES NN,
  received_at NN,
  initial_quantity NN,
  ...
)
```

- PK: primary key;
- FK: foreign key;
- UQ: unique;
- NN: not null.

The complete mapping is documented in `docs/relational_schema.md`.

12. Normalization without fear

12.1 Why normalization exists

Normalization organizes relations so each fact is stored in an appropriate place with controlled dependencies. Its practical goals are to reduce unnecessary duplication and prevent anomalies.

Imagine one giant sheet:

```
DonorCode, DonorBloodType, ConsentType, ConsentVersion,
EventCode, SampleCode, SampleTypeName, SampleUnit,
AliquotCode, StorageBox, ProjectCode, ProjectTitle,
ResearcherName, ResearcherEmail, QuantityUsed
```

Every usage row repeats donor, sample, project, and researcher facts.

12.2 The three anomalies

Update anomaly: a researcher changes institution. If it appears in 100 usage rows, every copy must be changed. Missing one produces contradiction.

Insert anomaly: a project cannot be recorded until it has sample usage, because project columns exist only in the giant usage sheet.

Delete anomaly: deleting the last usage row may erase the only recorded copy of the project and researcher. Normalization separates independent facts so one operation affects the correct thing.

12.3 Functional dependency

A functional dependency $X \rightarrow Y$ means: if two rows agree on X, they must agree on Y.

Examples:

```
donor_id -> donor_code, birth_year, donor_status
sample_id -> sample_code, sample_type_id, initial_quantity
project_id -> project_code, project_title, ethics_approval_code
(project_id, researcher_id) -> project_role, joined_on, left_on
```

X is the **determinant**. Functional dependency is about meaning and valid states, not coincidence in the current test rows.

12.4 First Normal Form (1NF)

A relation is in 1NF when:

- each column contains atomic values for the chosen design;
- there are no repeating column groups;
- rows can be identified.

Bad:

donor_code	consent_1	consent_2	consent_3

Bad:

donor_code	consent_types
BIO-D0001	research,genetics,commercial

Better:

donor_id	consent_type_id	version_no	status

One row represents one consent document/version.

“Atomic” depends on the system’s needs. A full name might be atomic if no query ever uses its components. In BioVault, direct names are excluded entirely.

12.5 Second Normal Form (2NF)

A relation is in 2NF when:

1. it is in 1NF; and
2. every non-key attribute depends on the **whole** candidate key, not part of a composite key.

Suppose we create:

```
PROJECT_RESEARCHERS(
  project_id,
  researcher_id,
  project_title,
  researcher_email,
  project_role
)
```

with composite key (project_id, researcher_id).

- project_title depends only on project_id;
- researcher_email depends only on researcher_id;
- project_role depends on the full pair.

Move project title to research_projects and email to researchers. Keep role in the bridge.

Tables with a single-column candidate key cannot have a partial dependency on that key, but they may still violate 3NF.

12.6 Third Normal Form (3NF)

A practical statement of 3NF:

1. the relation is in 2NF; and
2. non-key attributes do not depend on other non-key attributes.

Bad:

```
SAMPLES(
  sample_id,
  sample_type_id,
  sample_type_name,
  default_unit,
  ...
)
```

Dependencies:

```
sample_id -> sample_type_id
sample_type_id -> sample_type_name, default_unit
```

Therefore sample_id determines type name indirectly. This is a **transitive dependency**. Move type facts to sample_types.

BioVault applies the same reasoning to:

- consent names in consent_types;
- test names and result units in test_types;
- storage descriptions in storage_units;
- researcher facts in researchers;
- project facts in research_projects.

12.7 “The key, the whole key, and nothing but the key”

This sentence is a memory aid:

- the key — 1NF and meaningful identification;
- the whole key — no partial dependency (2NF);
- nothing but the key — no transitive dependency (3NF).

It is not a complete formal definition, but it is useful for beginner review.

12.8 Why the audit log contains JSONB

audit_log stores before/after row snapshots as JSONB. This is intentionally generic and denormalized.

Why is that acceptable?

- it is technical history, not operational master data;

- exact historical shape matters more than joining individual snapshot fields;
- it supports several audited tables through one mechanism;
- current authoritative data remains normalized.

Normalization is a design tool, not a rule that forbids every controlled copy. Denormalize only for a stated purpose and understand the trade-off.

12.9 Normalization lab

Given:

```
TEST_SHEET(  
  request_code,  
  sample_code,  
  sample_type_name,  
  test_code,  
  test_name,  
  test_result_unit,  
  requester_email,  
  requested_on,  
  result  
)
```

Identify at least four functional dependencies, two update anomalies, and the relations you would create to reach 3NF.

Part IV — Build the Database with PostgreSQL

13. PostgreSQL, tools, and project structure

13.1 Why PostgreSQL?

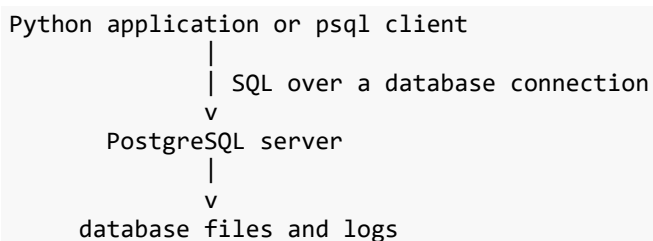
PostgreSQL is an open-source relational DBMS with strong support for:

- standards-based SQL;
- transactions and concurrency;
- constraints and referential integrity;
- advanced queries;
- functions and triggers;
- rich data types such as JSONB;
- mature administration and tooling.

BioVault targets PostgreSQL 16 or later. The core ideas transfer to other relational products, but some syntax changes by dialect.

13.2 Client and server

PostgreSQL normally uses a client/server architecture:



The **server** manages data, locks, transactions, and execution. A **client** sends SQL and displays results. `psql` is PostgreSQL's command-line client. The BioVault Python application is another client.

13.3 Connection string

The project uses:

```
postgresql://biobank:biobank_dev@localhost:55432/biobank
```

Read it as:

Segment	Meaning
postgresql://	protocol/driver family
biobank before :	database user
biobank_dev	password for this local demonstration
localhost	server is this computer
55432	published host port
final /biobank	database name

Never commit production credentials. The included password is for an isolated local educational container.

13.4 Why Docker is used

Docker runs PostgreSQL in a reproducible container. The same Compose definition can start the expected version and configuration on another machine.

```
docker compose up -d database
```

Useful commands:

```
# Initialize the schema and data.
docker compose exec -T database `
  psql -U biobank -d biobank -f /project/sql/setup.sql

# Run database acceptance tests.
docker compose exec -T database `
  psql -U biobank -d biobank -f /project/sql/test_constraints.sql

# Stop the container while retaining its named volume.
docker compose down
```

For a clean project-only rebuild:

```
docker compose down -v
docker compose up -d database
docker compose exec -T database `
  psql -U biobank -d biobank -f /project/sql/setup.sql
```

`down -v` deletes this Compose project's database volume. It is appropriate when you intentionally want a clean rebuild, not when you need to retain work.

13.5 Why the SQL is split into files

File	Responsibility
<code>create_tables.sql</code>	schema, tables, constraints, indexes
<code>triggers_procedures.sql</code>	functions and triggers
<code>load_data.sql</code>	deterministic synthetic data
<code>views.sql</code>	reusable read models
<code>queries.sql</code>	demonstration queries
<code>test_constraints.sql</code>	database acceptance tests
<code>setup.sql</code>	runs creation, logic, data, and views in order
<code>teardown.sql</code>	removes the project schema

Separation makes each concern reviewable while `setup.sql` preserves one-command reproducibility.

13.6 The `psql` safety line

Project scripts begin with:

```
\set ON_ERROR_STOP on
```

This is a `psql` command, not portable SQL. It tells the client to stop when a statement fails. Without it, a setup script might continue and leave a misleading partial installation.

Setup checkpoint

Explain the difference among:

1. the PostgreSQL server;
2. the biobank database;
3. the biobank schema;
4. the donors table;
5. the `psql` client.

14. DDL: create the structure

14.1 CREATE TABLE

DDL defines database objects:


```
CREATE TABLE donors (
  donor_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  donor_code VARCHAR(20) NOT NULL,
  donor_status VARCHAR(12) NOT NULL DEFAULT 'ACTIVE'
);
```

Read each column definition left to right:

1. name;
2. type;
3. column constraints or default.

Table-level constraints follow the column list.

14.2 Identity columns

```
BIGINT GENERATED BY DEFAULT AS IDENTITY
```

asks PostgreSQL to generate numeric identifiers. BY DEFAULT still permits an explicit value when controlled seed data needs stable IDs.

Applications should normally retrieve the generated value:

```
INSERT INTO donors (donor_code, sex_at_birth, registered_on)
VALUES ('BIO-D9001', 'UNKNOWN', CURRENT_DATE)
RETURNING donor_id;
```

Do not calculate $\text{MAX}(\text{id}) + 1$. Two concurrent transactions can calculate the same value.

14.3 Choosing text types

- VARCHAR(n) applies a maximum length.
- TEXT stores variable-length text without an application-level limit.
- CHAR(n) pads to fixed length and is rarely needed for ordinary codes.

Limits should reflect real requirements, not random numbers. A maximum also helps reject obviously malformed input.

14.4 Exact numeric data

BioVault uses:

```
NUMERIC(14,3)
```

This allows 14 total decimal digits, 3 after the decimal point. Exact decimal arithmetic avoids binary floating-point surprises in controlled quantities.

Do not confuse precision and scale:

```
NUMERIC(14,3)
      ^  ^
      |  |
      |  | digits after decimal
      |  | total digits
```

14.5 DATE and TIMESTAMPTZ

Use DATE when the business meaning is a calendar date:

- project start date;
- consent expiry date;
- usage date.

Use TIMESTAMPTZ when the exact instant matters:

- sample received time;
- collection time;
- audit time.

PostgreSQL stores TIMESTAMPTZ as an instant and converts display according to the session time zone. The type name does not mean it preserves the original typed zone label.

14.6 Defaults

```
donor_status VARCHAR(12) NOT NULL DEFAULT 'ACTIVE',
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
```

A default supplies a value when the insert omits that column. It does not override an explicitly supplied value, and it does not validate whether the default makes sense for every workflow.

14.7 Rebuild strategy

The educational creation script begins:

```
DROP SCHEMA IF EXISTS biobank CASCADE;
CREATE SCHEMA biobank;
```

This deliberately destroys and recreates the project schema for deterministic testing. It is appropriate for a disposable development database. A production system would use versioned migrations and backups, not drop the live schema on startup.

Common DDL mistakes

- using a text column for dates or quantities;
- forgetting a primary key;
- using nullable columns when the fact is required;
- relying only on application validation;
- dropping broad objects without confirming the environment;
- mixing creation, demonstration edits, and tests in one unrepeatable file.

15. Constraints: make invalid states difficult

15.1 NOT NULL

```
sample_code VARCHAR(24) NOT NULL
```

The sample code is required. NOT NULL is clearer and stronger than hoping every client supplies the value.

15.2 UNIQUE

```
CONSTRAINT uq_samples_code UNIQUE (sample_code)
```

No two sample rows may have the same sample code. A unique constraint also documents a candidate key. BioVault uses a composite uniqueness rule:

```
CONSTRAINT uq_aliquots_position
    UNIQUE (storage_unit_id, position_code)
```

Position A01 can exist in many boxes, but only once inside the same box.

15.3 CHECK

A check constraint requires an expression to be true or unknown for each row:

```
CONSTRAINT ck_aliquots_quantities CHECK (
    initial_quantity > 0
    AND current_quantity >= 0
    AND current_quantity <= initial_quantity
)
```

Another check limits controlled status values:

```
CHECK (sample_status IN (
    'AVAILABLE', 'RESERVED', 'DEPLETED', 'DESTROYED'
))
```

And PostgreSQL regular-expression syntax validates codes:

```
CHECK (donor_code ~ '^BIO-D[0-9]{4}$')
```

The anchors ^ and \$ require the complete value to match.

15.4 Checks and NULL

A check rejects FALSE, not UNKNOWN. If a value must exist, pair the check with NOT NULL.

For example:

```
birth_year SMALLINT,
CHECK (birth_year IS NULL OR birth_year BETWEEN 1900 AND 2100)
```

permits an unknown year. That is intentional. A required quantity uses both NOT NULL and a positive check.

15.5 Foreign keys

```
CONSTRAINT fk_samples_collection_event
    FOREIGN KEY (collection_event_id)
    REFERENCES collection_events(collection_event_id)
ON DELETE RESTRICT
```

This enforces referential integrity. It does not automatically create an index on the child column in PostgreSQL, so expected join and delete-check paths may need explicit indexes.

15.6 Constraint naming

Names such as ck_aliquots_quantities and fk_samples_collection_event make errors and schema reviews understandable.

One naming convention:

- pk_ — primary key;
- fk_ — foreign key;
- uq_ — unique;
- ck_ — check;
- idx_ — index;
- trg_ — trigger;
- fn_ — internal function.

15.7 Constraint versus trigger

Prefer a constraint when it can express the rule. A constraint is declarative, visible to tools, and usually easier to reason about.

Use a trigger when the rule:

- spans several tables;
- performs a related automatic action;
- requires transactional locking;
- needs old and new row values for an audit.

Do not use a trigger just to check `quantity > 0`.

15.8 Negative testing

A database test should deliberately attempt invalid data:

```
DO $$
BEGIN
```

```

BEGIN
  INSERT INTO donors (
    donor_code, sex_at_birth, donor_status, registered_on
  )
  VALUES ('BAD-CODE', 'UNKNOWN', 'ACTIVE', CURRENT_DATE);

  RAISE EXCEPTION 'Expected malformed code to fail';
EXCEPTION
  WHEN check_violation THEN
    RAISE NOTICE 'PASS: malformed code rejected';
END;
END;
$$;

```

A successful happy-path insert proves only that some data works. A rejection test proves the protection works.

16. DML: insert, update, delete

16.1 INSERT

Always name columns:

```

INSERT INTO donors (
  donor_code,
  sex_at_birth,
  birth_year,
  donor_status,
  registered_on
)
VALUES (
  'BIO-D9999',
  'UNKNOWN',
  NULL,
  'ACTIVE',
  CURRENT_DATE
)
RETURNING donor_id, donor_code;

```

Naming columns makes the statement readable and safer if table order changes.

16.2 Load parents before children

Foreign keys determine seed order:

```

donors before consents and collection_events
researchers before projects
projects and researchers before project_researchers
collection_events and sample_types before samples
samples and storage_units before aliquots

```

This is a topological ordering of dependencies.

16.3 UPDATE

```

UPDATE donors
SET donor_status = 'INACTIVE'
WHERE donor_code = 'BIO-D9999'
RETURNING donor_id, donor_status;

```

Before pressing Enter, read the WHERE clause aloud. Without it, every row is updated.

A safe habit:

```

SELECT *
FROM donors
WHERE donor_code = 'BIO-D9999';

```

Verify the target first, then perform the update in a transaction.

16.4 DELETE

```
DELETE FROM donors
WHERE donor_code = 'BIO-D9999'
RETURNING donor_id, donor_code;
```

Foreign keys may reject deletion when the donor has traceable scientific history. That rejection is a feature. For lifecycle records, a status such as INACTIVE or WITHDRAWN is often safer than physical deletion.

16.5 A reversible demonstration

queries.sql demonstrates DML without polluting the seed:

```
BEGIN;

INSERT INTO donors (...) VALUES (...);
UPDATE donors SET donor_status = 'INACTIVE' WHERE ...;
DELETE FROM donors WHERE ...;

ROLLBACK;
```

The statements really execute, constraints and triggers really run, but the final rollback restores the starting state.

16.6 Idempotent and deterministic scripts

- **Deterministic:** the same input and environment produce the expected result.
- **Idempotent:** repeated execution has the same final effect as one execution.

The whole BioVault setup is repeatable because it recreates its development schema. The demonstration transaction is repeatable because it rolls back.

Production migrations require different patterns, but repeatability remains essential.

17. Seed data that teaches and tests

17.1 Test data is part of the design

Ten identical “active” rows prove little. Good synthetic data represents important states and relationships.

BioVault includes:

- active, inactive, withdrawn, and deceased donor states;
- active, expired, and withdrawn consents;
- multiple sample types and measurement units;
- accepted, pending, quarantined, and rejected quality;
- available, reserved, exhausted, and destroyed inventory;
- requested, in-progress, completed, and cancelled tests;
- active, completed, planned, and suspended projects;
- hierarchical storage at several temperatures.

17.2 Main-table row counts

Table	Seed rows
Donors	12
Researchers	10
Research projects	10
Project memberships	20
Collection events	12
Samples	20
Storage units	14

Table	Seed rows
Aliquots	20
Test requests	15
Sample usage	12

Lookup tables contain the number of meaningful controlled values, not padded duplicates. “At least ten” makes sense for main operational entities, but not for inventing ten fake blood-test categories when six are required.

17.3 Synthetic does not mean random nonsense

Synthetic data should:

- contain no real personal data;
- obey foreign keys and business rules;
- use realistic formats and ranges;
- include boundary cases;
- remain easy to explain.

The seed IDs are explicit where stable relationships make the educational data easier to read. This is acceptable in a deterministic development rebuild.

17.4 Verify counts

```
SELECT 'donors' AS table_name, COUNT(*) AS row_count FROM donors
UNION ALL
SELECT 'samples', COUNT(*) FROM samples
UNION ALL
SELECT 'aliquots', COUNT(*) FROM aliquots;
```

Row counts prove minimum coverage, not semantic correctness. Combine them with constraint and workflow tests.

Part V — Query SQL from First SELECT to Advanced Analytics

18. Basic SELECT

18.1 The query pipeline

A basic query:

```
SELECT donor_code, blood_type
FROM donors
WHERE donor_status = 'ACTIVE'
ORDER BY donor_code;
```

Read it in this conceptual order:

1. FROM — choose source rows;
2. WHERE — filter rows;
3. SELECT — choose or calculate output columns;
4. ORDER BY — sort the result.

SQL is written with SELECT first, but understanding the logical order explains many alias and aggregation rules.

18.2 Select specific columns

Prefer:

```
SELECT donor_code, donor_status
FROM donors;
```

over SELECT * in application code because explicit columns:

- document the dependency;
- avoid transferring unused data;
- keep output stable when schema columns are added;
- prevent accidental exposure of sensitive fields.

SELECT * is convenient for quick interactive inspection.

18.3 Aliases

```
SELECT
    donor_code AS code,
    donor_status AS status
FROM donors AS d;
```

Aliases improve result labels and shorten qualified names. Use meaningful aliases in complex queries; collection_events AS ce is clearer than arbitrary letters in a six-table join.

18.4 Filtering with WHERE

```
WHERE quality_status = 'ACCEPTED'
AND sample_status = 'AVAILABLE'
```

Common operators:

Operator	Meaning
=	equal
<>	not equal
<, <=, >, >=	comparison
BETWEEN	inclusive range
IN (...)	equals any listed value
LIKE	pattern match
IS NULL	null test
AND, OR, NOT	logical combination

Operator	Meaning
----------	---------

Use parentheses when mixing AND and OR:

```
WHERE quality_status = 'ACCEPTED'
      AND (sample_status = 'AVAILABLE' OR sample_status = 'RESERVED')
```

18.5 Pattern matching

```
WHERE sample_code LIKE 'SMP-000%'
```

- % matches zero or more characters;
- _ matches exactly one character.

PostgreSQL ILIKE performs case-insensitive matching:

```
WHERE sample_type ILIKE '%plasma%'
```

18.6 DISTINCT

```
SELECT DISTINCT quantity_unit
FROM aliquots;
```

DISTINCT removes duplicate output rows. Do not use it to hide a bad join. If a row appears more times than expected, first investigate relationship cardinality.

18.7 CASE

```
SELECT
  aliquot_code,
  current_quantity,
  CASE
    WHEN current_quantity = 0 THEN 'EMPTY'
    WHEN current_quantity <= initial_quantity * 0.25 THEN 'LOW'
    ELSE 'ADEQUATE'
  END AS stock_level
FROM aliquots;
```

CASE adds conditional logic to a query result without changing stored data.

Basic query lab

Write queries to:

1. list withdrawn donors;
2. list samples received after a chosen date;
3. show available aliquots with less than 25% remaining;
4. display all test requests whose text or numeric result is missing.

19. Joins: reconnect normalized facts

19.1 Why joins are necessary

Normalization stores each fact once. A query reconnects related facts through keys.

To show sample code and type name:

```
SELECT s.sample_code, st.type_name
FROM samples AS s
JOIN sample_types AS st
  ON st.sample_type_id = s.sample_type_id;
```

The ON condition states how rows correspond.

19.2 Inner join

JOIN means INNER JOIN by default. It keeps matching row combinations.

```
SELECT s.sample_code, ce.event_code, d.donor_code
FROM samples AS s
JOIN collection_events AS ce
  ON ce.collection_event_id = s.collection_event_id
JOIN donors AS d
  ON d.donor_id = ce.donor_id;
```

Notice that donor identity is reached through the collection event. It is not duplicated in samples.

19.3 Multi-table lineage join

The project query connects six tables:

```
SELECT
  d.donor_code,
  ce.event_code,
  s.sample_code,
  st.type_name,
  a.aliquot_code,
  su.location_code
FROM donors AS d
JOIN collection_events AS ce ON ce.donor_id = d.donor_id
JOIN samples AS s ON s.collection_event_id = ce.collection_event_id
JOIN sample_types AS st ON st.sample_type_id = s.sample_type_id
JOIN aliquots AS a ON a.sample_id = s.sample_id
JOIN storage_units AS su ON su.storage_unit_id = a.storage_unit_id
ORDER BY d.donor_code, s.sample_code;
```

Build long joins one relationship at a time. After adding each join, inspect row counts and a few keys.

19.4 Left join

A LEFT JOIN keeps every row from the left side, even when the right side has no match:

```
SELECT
  st.type_name,
  COUNT(s.sample_id) AS sample_count
FROM sample_types AS st
LEFT JOIN samples AS s
  ON s.sample_type_id = st.sample_type_id
GROUP BY st.sample_type_id, st.type_name;
```

Types with no samples appear with count zero.

19.5 A subtle outer-join mistake

This query accidentally removes unmatched rows:

```
SELECT ...
FROM sample_types st
LEFT JOIN samples s ON s.sample_type_id = st.sample_type_id
WHERE s.sample_status = 'AVAILABLE';
```

For an unmatched row, s.sample_status is null, so WHERE removes it. If the intention is to keep every type, move the condition into ON:

```
LEFT JOIN samples s
  ON s.sample_type_id = st.sample_type_id
  AND s.sample_status = 'AVAILABLE'
```

19.6 Self join

A self join uses the same table twice:

```
SELECT
    child.unit_name,
    parent.unit_name AS parent_name
FROM storage_units AS child
LEFT JOIN storage_units AS parent
    ON parent.storage_unit_id = child.parent_storage_unit_id;
```

Aliases distinguish the child role from the parent role.

19.7 Join multiplication

If a sample has two aliquots and three test requests, joining both child tables directly produces six combinations for that sample. Aggregates may be inflated.

Solutions depend on the question:

- aggregate each child before joining;
- use `COUNT(DISTINCT ...)`;
- query children separately;
- use correlated subqueries or lateral joins when appropriate.

Never add `DISTINCT` without understanding why duplicates appeared.

Join lab

Create a report showing test request code, sample code, donor code, test name, requester name, project code if present, and request status. Decide which join must be left-outer and explain why.

20. Aggregation, grouping, and HAVING

20.1 Aggregate functions

Aggregate functions turn several rows into a summary:

- `COUNT(*)`;
- `COUNT(column)`;
- `SUM`;
- `AVG`;
- `MIN`;
- `MAX`.

`COUNT(*)` counts rows. `COUNT(column)` counts non-null values in that column.

20.2 GROUP BY

```
SELECT
    st.type_name,
    COUNT(DISTINCT s.sample_id) AS sample_count,
    COUNT(a.aliquot_id) AS aliquot_count,
    COALESCE(SUM(a.current_quantity), 0) AS quantity_remaining
FROM sample_types AS st
LEFT JOIN samples AS s ON s.sample_type_id = st.sample_type_id
LEFT JOIN aliquots AS a ON a.sample_id = s.sample_id
GROUP BY st.sample_type_id, st.type_name
ORDER BY sample_count DESC;
```

Every selected expression must either:

- be aggregated; or
- identify the group.

Grouping by the primary key and displayed name communicates both identity and output.

20.3 COALESCE

SUM of no matching values returns null. COALESCE chooses the first non-null expression:

```
COALESCE(SUM(a.current_quantity), 0)
```

Do not automatically turn every null into zero. Here, zero remaining quantity is a meaningful representation of no matching stock. In other domains, unknown and zero may be different.

20.4 WHERE versus HAVING

- WHERE filters individual rows before grouping.
- HAVING filters groups after aggregation.

```
SELECT
    p.project_code,
    COUNT(DISTINCT ce.donor_id) AS donors_used
FROM research_projects AS p
JOIN sample_usage AS u ON u.project_id = p.project_id
JOIN aliquots AS a ON a.aliquot_id = u.aliquot_id
JOIN samples AS s ON s.sample_id = a.sample_id
JOIN collection_events AS ce
    ON ce.collection_event_id = s.collection_event_id
GROUP BY p.project_id, p.project_code
HAVING COUNT(DISTINCT ce.donor_id) > 1;
```

HAVING is required because the filter depends on the group count.

20.5 Filtered aggregates

PostgreSQL supports:

```
COUNT(*) FILTER (WHERE tr.request_status = 'COMPLETED')
```

and:

```
AVG(tr.completed_on - tr.requested_on)
    FILTER (WHERE tr.request_status = 'COMPLETED')
```

This can express several conditional measures in one grouped query more clearly than repeated CASE expressions.

Aggregation lab

Produce one row per project with:

- project code;
- number of members;
- number of usage events;
- total quantity used;
- date of latest use.

Think carefully about joining two one-to-many children before aggregating.

21. Subqueries and EXISTS

21.1 Scalar subquery

A scalar subquery returns one value:

```
SELECT aliquot_code, current_quantity
FROM aliquots
WHERE current_quantity > (
    SELECT AVG(current_quantity)
    FROM aliquots
);
```

If it returns more than one row, PostgreSQL raises an error.

21.2 Correlated subquery

This project query compares an aliquot with the average for its own unit:

```
SELECT a.aliquot_code, a.current_quantity, a.quantity_unit
FROM aliquots AS a
WHERE a.current_quantity > (
    SELECT AVG(a2.current_quantity)
    FROM aliquots AS a2
    WHERE a2.quantity_unit = a.quantity_unit
);
```

The inner query references `a.quantity_unit` from the outer row, so it is correlated.

Conceptually, evaluate the inner query for the current outer unit. PostgreSQL's optimizer may execute an equivalent, more efficient strategy.

21.3 IN and EXISTS

IN asks whether a value appears in a set:

```
WHERE donor_id IN (
    SELECT donor_id FROM consents WHERE consent_status = 'ACTIVE'
)
```

EXISTS asks whether at least one matching row exists:

```
WHERE EXISTS (
    SELECT 1
    FROM consents AS c
    WHERE c.donor_id = d.donor_id
    AND c.consent_status = 'ACTIVE'
)
```

The `SELECT 1` is a convention; the selected value is irrelevant. `EXISTS` stops being logically concerned after a match is found.

21.4 Two independent existence rules

BioVault finds donors who both:

1. have an accepted sample; and
2. have current active research consent.

```
SELECT d.donor_code
FROM donors AS d
WHERE EXISTS (
    SELECT 1
    FROM collection_events AS ce
    JOIN samples AS s
    ON s.collection_event_id = ce.collection_event_id
    WHERE ce.donor_id = d.donor_id
    AND s.quality_status = 'ACCEPTED'
)
AND EXISTS (
    SELECT 1
    FROM consents AS c
    JOIN consent_types AS ct
    ON ct.consent_type_id = c.consent_type_id
    WHERE c.donor_id = d.donor_id
    AND ct.consent_code = 'RESEARCH_USE'
    AND c.consent_status = 'ACTIVE'
    AND (c.expires_on IS NULL OR c.expires_on >= CURRENT_DATE)
);
```

Separate `EXISTS` blocks mirror two separate requirements.

21.5 NOT IN and null danger

If a NOT IN subquery contains null, the result may become unknown for every comparison. NOT EXISTS is often safer:

```
SELECT d.donor_code
FROM donors AS d
WHERE NOT EXISTS (
    SELECT 1
    FROM collection_events AS ce
    WHERE ce.donor_id = d.donor_id
);
```

This returns donors with no collection event.

22. Common Table Expressions and recursion

22.1 CTE

A Common Table Expression gives a query block a name:

```
WITH active_projects AS (
    SELECT project_id, project_code
    FROM research_projects
    WHERE project_status = 'ACTIVE'
)
SELECT *
FROM active_projects
ORDER BY project_code;
```

Use CTEs to:

- make multi-stage logic readable;
- reuse an intermediate result within a statement;
- isolate aggregation stages;
- express recursion.

A CTE is not automatically faster. Modern PostgreSQL may inline it, and materialization choices depend on version and syntax.

22.2 Recursive CTE structure

A recursive CTE has:

1. an anchor query;
2. UNION ALL;
3. a recursive query that references the CTE.

BioVault reconstructs storage paths:

```
WITH RECURSIVE storage_tree AS (
    SELECT
        storage_unit_id,
        parent_storage_unit_id,
        unit_name,
        unit_name::TEXT AS full_path,
        1 AS depth
    FROM storage_units
    WHERE parent_storage_unit_id IS NULL

    UNION ALL

    SELECT
        child.storage_unit_id,
        child.parent_storage_unit_id,
        child.unit_name,
        parent.full_path || ' > ' || child.unit_name,
```

```

        parent.depth + 1
    FROM storage_units AS child
    JOIN storage_tree AS parent
        ON parent.storage_unit_id = child.parent_storage_unit_id
)
SELECT storage_unit_id, depth, full_path
FROM storage_tree
ORDER BY full_path;

```

22.3 Think through one iteration

Anchor:

Central Biobank

First recursive step:

Central Biobank > Cryostorage Room

Next:

Central Biobank > Cryostorage Room > Freezer F01

The process continues until no child rows match.

22.4 Cycle protection

The schema prevents a unit from being its own immediate parent, but it does not by itself prevent a longer cycle such as $A \rightarrow B \rightarrow C \rightarrow A$.

A production hierarchy can add:

- a trigger that checks ancestors before changing a parent;
- a recursive query with a visited-ID path;
- PostgreSQL CYCLE syntax where appropriate;
- application restrictions and tests.

This is a good example of identifying a limitation honestly instead of claiming the model enforces more than it does.

23. Window functions

23.1 Aggregate versus window

An aggregate with GROUP BY collapses rows. A **window function** calculates across related rows while preserving one output row per input row.

```

SELECT
    project_code,
    usage_events,
    DENSE_RANK() OVER (ORDER BY usage_events DESC) AS usage_rank
FROM v_project_usage_summary;

```

Each project remains visible, and each receives a rank.

23.2 The window definition

```

function(...) OVER (
    PARTITION BY ...
    ORDER BY ...
    ROWS BETWEEN ...
)

```

- PARTITION BY creates independent groups;
- ORDER BY establishes sequence inside a group;
- the frame defines which nearby rows participate.

23.3 Ranking functions

Suppose usage counts are 5, 5, 3.

Function	Ranks
ROW_NUMBER()	1, 2, 3
RANK()	1, 1, 3
DENSE_RANK()	1, 1, 2

BioVault uses DENSE_RANK because tied projects share a rank without leaving a gap.

23.4 Running total example

```
SELECT
  project_id,
  used_on,
  quantity_used,
  SUM(quantity_used) OVER (
    PARTITION BY project_id
    ORDER BY used_on, usage_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_quantity
FROM sample_usage;
```

The explicit frame makes a row-by-row cumulative total. Adding usage_id gives a deterministic tie-breaker when several uses share a date.

23.5 Previous value with LAG

```
SELECT
  aliquot_id,
  used_on,
  quantity_used,
  LAG(quantity_used) OVER (
    PARTITION BY aliquot_id
    ORDER BY used_on, usage_id
  ) AS previous_quantity_used
FROM sample_usage;
```

Window functions are central to analytics, but they do not replace ordinary grouping or joins. Choose the tool that matches the desired result shape.

Part VI — Reusable SQL, Performance, and Transactional Correctness

24. Views: saved relational questions

24.1 What is a view?

A standard SQL view stores a query definition under a name:

```
CREATE OR REPLACE VIEW v_available_inventory AS
SELECT ...
FROM ...
WHERE ...;
```

Query it like a table:

```
SELECT aliquot_code, sample_code, current_quantity
FROM v_available_inventory
WHERE sample_type ILIKE '%plasma%';
```

The standard view normally does not store a separate copy of rows. PostgreSQL expands its definition into the surrounding query.

24.2 Why use a view?

A view can:

- hide repeated join complexity;
- give the interface a stable read model;
- standardize definitions such as “available inventory”;
- expose selected columns instead of base-table details;
- simplify reporting permissions.

BioVault includes:

1. v_available_inventory;
2. v_project_usage_summary;
3. v_consent_monitor.

24.3 View versus table

Table	Standard view
stores base rows	stores query definition
accepts ordinary direct writes	may be read-only or conditionally updatable
owns constraints	depends on base-table constraints
persists data independently	reflects current underlying data

A **materialized view** is different: it stores a query result and must be refreshed. BioVault does not need one for the small operational dataset.

24.4 View design

A useful operational view should have a clear grain.

For v_available_inventory, the grain is:

one row per currently available aliquot with positive quantity.

Knowing the grain prevents accidental double counting.

Do not put an ORDER BY inside a general view and assume every consumer receives that order. Order belongs to the outer query.

24.5 Security note

A view can reduce exposure, but it is not automatically a complete security boundary. PostgreSQL privileges, view ownership/security options, row-level security, and application authorization all require deliberate design.

25. Indexes and query planning

25.1 The book-index analogy

Without an index, finding every mention of “consent” in a book may require reading every page. A back-of-book index points to likely pages.

A database index is an auxiliary structure that helps locate rows. PostgreSQL’s common default is a B-tree index, useful for equality, ranges, and ordering.

25.2 The cost of an index

Indexes are not free:

- they use disk space;
- inserts, updates, and deletes must maintain them;
- too many can slow writes;
- an unused index adds cost without value.

Create indexes for demonstrated access paths, not every column.

25.3 Constraint-created indexes

PostgreSQL automatically creates a unique B-tree index for a primary key or unique constraint.

PostgreSQL does **not** automatically index the child side of a foreign key. BioVault explicitly adds useful foreign-key indexes, such as:

```
CREATE INDEX idx_samples_collection_event
ON samples (collection_event_id);
```

25.4 Composite index order

```
CREATE INDEX idx_consents_donor_status_dates
ON consents (
    donor_id,
    consent_status,
    granted_on,
    expires_on
);
```

Column order matters. The leading columns support queries that first narrow by donor and status, then evaluate dates.

An index on (a, b) is not generally equivalent to separate indexes on a and b, and it is not as naturally useful for a predicate on only b.

25.5 Partial index

BioVault indexes only available aliquots:

```
CREATE INDEX idx_aliquots_availability
ON aliquots (aliquot_status, current_quantity)
WHERE aliquot_status = 'AVAILABLE';
```

The predicate makes the index smaller when unavailable historical rows are common. A query must logically match the predicate for PostgreSQL to use it.

Because `aliquot_status` is constant inside the partial index, an alternative design could index only `current_quantity` under the same predicate. The submitted version remains readable and supports the demonstrated access path; the alternative is a reasonable optimization discussion in a defense.

25.6 EXPLAIN

```
EXPLAIN
SELECT *
FROM aliquots
WHERE aliquot_status = 'AVAILABLE'
AND current_quantity > 0;
```

EXPLAIN shows the planned operations and estimated rows/costs.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT ...;
```

actually executes the query and reports real timing and buffer use. Do not run ANALYZE casually on destructive DML; it performs the statement.

25.7 Sequential scan is not automatically bad

For a table with 20 rows, reading the whole table may be faster than using an index. The optimizer considers table size, selectivity, statistics, and cost.

An index is justified by the expected production access path even if the tiny academic seed does not always use it.

25.8 Sargability

A predicate is informally called **sargable** when an index can be used effectively to locate matching rows.

This may be less index-friendly:

```
WHERE LOWER(donor_code) = 'bio-d0001'
```

Alternatives include:

- normalize stored codes to uppercase and compare directly;
- create an expression index on LOWER(donor_code);
- use a case-insensitive type/extension when justified.

Choose the design based on actual requirements and measured plans.

Performance checkpoint

Explain why creating an index on every column can make the system slower even though indexes speed some reads.

26. Transactions and ACID

26.1 What is a transaction?

A **transaction** is a logical unit of work. It either commits as a whole or rolls back.

```
BEGIN;
UPDATE aliquots SET current_quantity = current_quantity - 0.050 WHERE ...;
INSERT INTO sample_usage (...) VALUES (...);
COMMIT;
```

If the insert fails after the update and no transaction protects the pair, inventory can change without a usage record.

26.2 ACID

Atomicity: all transaction operations happen or none remain.

Consistency: a successful transaction moves the database from one valid state to another, assuming rules are correctly designed.

Isolation: concurrent transactions behave according to defined visibility and interference rules.

Durability: once committed, changes survive ordinary failures according to the DBMS durability configuration. ACID is not a spell that fixes bad logic. The transaction must include all related operations, and the schema must express the correct constraints.

26.3 COMMIT and ROLLBACK

```
BEGIN;  
-- tentative work  
COMMIT;
```

makes work permanent.

```
BEGIN;  
-- tentative work  
ROLLBACK;
```

discards it.

Many client libraries automatically begin a transaction and require an explicit or context-managed commit.

26.4 Savepoints

```
BEGIN;  
SAVEPOINT before_optional_step;  
  
-- attempt optional work  
  
ROLLBACK TO SAVEPOINT before_optional_step;  
COMMIT;
```

A savepoint allows partial recovery inside a transaction. It does not make unrelated operations belong in one giant transaction.

26.5 Keep transactions focused

Long transactions can:

- retain locks;
- delay cleanup of old row versions;
- increase contention;
- make failures more expensive.

Collect user input before beginning the transaction. Start it when database work is ready, perform the smallest correct unit, and finish promptly.

27. Concurrency and row locks

27.1 The lost-stock race

Suppose aliquot A has 0.100 mL.

Two sessions request 0.080 mL at the same time:

```
Session 1 reads 0.100  
Session 2 reads 0.100  
Session 1 decides 0.080 is allowed  
Session 2 decides 0.080 is allowed  
Both record usage
```

Total requested is 0.160, greater than stock. A check performed only in an interface cannot prevent this.

27.2 Row-level locking

BioVault's trigger reads the aliquot with:

```
SELECT ...
FROM aliquots AS a
...
WHERE a.aliquot_id = NEW.aliquot_id
FOR UPDATE OF a;
```

The first transaction locks the target aliquot row. A second transaction trying to lock it waits. After the first commits and deducts stock, the second sees the new quantity and fails the overdraw check.

The lock is precise: it protects the inventory item being changed, not the whole table.

27.3 Why “check then update” must share a transaction

Locking is useful only when:

- the row is locked before the decision;
- validation and deduction occur in the same transaction;
- the lock lasts until commit/rollback.

Releasing the transaction between check and update reopens the race.

27.4 Isolation levels

PostgreSQL supports:

- READ COMMITTED — each statement sees committed data as of its start;
- REPEATABLE READ — a transaction sees a stable snapshot, with serialization rules for conflicting updates;
- SERIALIZABLE — strongest isolation, may abort transactions that cannot be serialized safely.

Row locks solve the specific aliquot-consumption conflict under the default READ COMMITTED behavior. Higher isolation is not a substitute for understanding the exact invariant, and applications must be prepared to retry serialization failures when using SERIALIZABLE.

27.5 Deadlocks

A deadlock can occur when:

```
Transaction A locks aliquot 1, then wants aliquot 2.
Transaction B locks aliquot 2, then wants aliquot 1.
```

PostgreSQL detects the cycle and aborts one transaction. Reduce deadlock risk by locking multiple resources in a consistent order and keeping transactions short. Applications should handle retryable transaction failures.

28. Functions, procedures, and triggers

28.1 Function

A PostgreSQL function accepts parameters and returns a value or set:

```
CREATE OR REPLACE FUNCTION record_sample_usage(...)
RETURNS BIGINT
LANGUAGE plpgsql
AS $$
...
$$;
```

Call it with:

```
SELECT record_sample_usage(
  1, 1, 1, DATE '2026-06-15', 0.050,
  'Authorized assay'
);
```

The return value is the new usage_id.

28.2 Procedure

A procedure is called with CALL and, in PostgreSQL, has different transaction capabilities and return conventions. BioVault uses a function because returning the generated usage ID fits a function naturally.

28.3 Trigger

A trigger runs automatically in response to a table event.

```
CREATE TRIGGER trg_validate_and_apply_sample_usage
BEFORE INSERT ON sample_usage
FOR EACH ROW
EXECUTE FUNCTION fn_validate_and_apply_sample_usage();
```

Vocabulary:

- **timing:** BEFORE, AFTER, or INSTEAD OF;
- **event:** INSERT, UPDATE, DELETE, sometimes TRUNCATE;
- **level:** once per statement or once per row;
- **NEW:** proposed/new row for insert/update;
- **OLD:** previous row for update/delete;
- **TG_OP, TG_TABLE_NAME:** trigger context.

28.4 Why use a public function plus a trigger?

The function provides a clear application API:

```
record_sample_usage(...)
```

The trigger protects the table even if someone inserts directly into sample_usage. It centralizes:

1. row locking;
2. stock validation;
3. project-date and status validation;
4. project membership validation;
5. consent validation;
6. inventory deduction.

A stricter production permission design could revoke direct inserts and grant only controlled function execution. The trigger remains defense in depth.

28.5 Trigger workflow

```
Application calls record_sample_usage
      |
      v
Function inserts sample_usage
      |
      v
BEFORE INSERT trigger fires
      |
      +--> lock aliquot
      +--> find donor lineage
      +--> check available quantity/status
      +--> check project authorization
      +--> check researcher membership
      +--> check active consent
      +--> deduct quantity
      |
      v
Insert completes and function returns usage_id
      |
      v
Transaction commits
```

Any exception aborts the statement and, unless handled, the transaction. The deduction does not remain without the usage row.

28.6 Detailed business-rule logic

The trigger first retrieves the aliquot and donor lineage:

```
SELECT
  a.current_quantity,
  a.aliquot_status,
  ce.donor_id
INTO
  v_available_quantity,
  v_aliquot_status,
  v_donor_id
FROM aliquots AS a
JOIN samples AS s ON s.sample_id = a.sample_id
JOIN collection_events AS ce
  ON ce.collection_event_id = s.collection_event_id
WHERE a.aliquot_id = NEW.aliquot_id
FOR UPDATE OF a;
```

It rejects:

- unknown aliquot;
- non-available status;
- quantity greater than remaining;
- unauthorized project date/status;
- missing membership on the usage date;
- missing active research consent on the usage date.

Then it deducts:

```
UPDATE aliquots
SET current_quantity = current_quantity - NEW.quantity_used,
    aliquot_status = CASE
      WHEN current_quantity - NEW.quantity_used = 0
      THEN 'EXHAUSTED'
      ELSE aliquot_status
    END
WHERE aliquot_id = NEW.aliquot_id;
```

Inside an UPDATE, the right-hand expressions use the old row values for that statement, so the calculation consistently derives both new columns.

28.7 Trigger risks

Triggers are powerful but can become invisible complexity:

- hidden side effects surprise developers;
- several triggers may interact;
- debugging can be harder;
- bulk loading may be slower;
- recursive trigger behavior can be dangerous.

Use them for clear invariants, name them well, document them, and test both success and rejection paths.

29. Auditing with OLD, NEW, and JSONB

29.1 Audit objective

An audit record should answer:

- which table and record changed?
- was it an update or delete?
- what were the old and new values?

- who performed the database operation?
- when did it happen?

29.2 Generic audit row

```
audit_id
table_name
record_id
operation
old_values JSONB
new_values JSONB
changed_by
changed_at
```

The trigger uses:

```
to_jsonb(OLD)
to_jsonb(NEW)
```

For an update, both snapshots are stored. For a delete, the old snapshot is stored and there is no new row.

29.3 Why AFTER?

Audit triggers use AFTER UPDATE OR DELETE because they record a change after the row operation has passed its checks. The audit insert remains in the same transaction: if the transaction later rolls back, its audit row rolls back too.

This records committed database history, not failed attempts. Capturing failed security attempts requires an external or separately durable logging design.

29.4 CURRENT_USER

changed_by DEFAULT CURRENT_USER records the PostgreSQL role executing the operation. If every application user shares one database role, this identifies the application account, not necessarily the human.

A production design can:

- use distinct database roles;
- set a verified session context;
- include authenticated application-user identity;
- forward audit logs to append-only external storage.

29.5 Audit is not backup

An audit log explains row-level changes. A backup restores data after loss. Neither replaces the other.

30. PostgreSQL views and logic: a guided project reading

When reading an unfamiliar SQL project, use this order:

1. docs/business_rules.md — understand required truth.
2. diagrams/ERD.png — understand concepts and relationships.
3. sql/create_tables.sql — see physical enforcement.
4. sql/triggers_procedures.sql — see multi-row behavior.
5. sql/load_data.sql — see realistic states.
6. sql/views.sql — see reusable read models.
7. sql/queries.sql — see questions answered.
8. sql/test_constraints.sql — see claims proved.

For every database object, ask:

- What real-world fact does this represent?
- What is the grain of one row?
- Which columns identify it?

- What invalid state does each constraint block?
- Who writes it?
- Who reads it?
- How is it tested?

This method turns a large schema from “many lines of SQL” into a collection of explainable decisions.

Part VII — Build the Bonus CRUD Application

31. CRUD and application architecture

31.1 What CRUD means

CRUD is a memory aid for four persistent operations:

CRUD	SQL
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

A real bonus application must connect to a database. Buttons that change only an in-memory list do not demonstrate database CRUD.

31.2 BioVault application layers

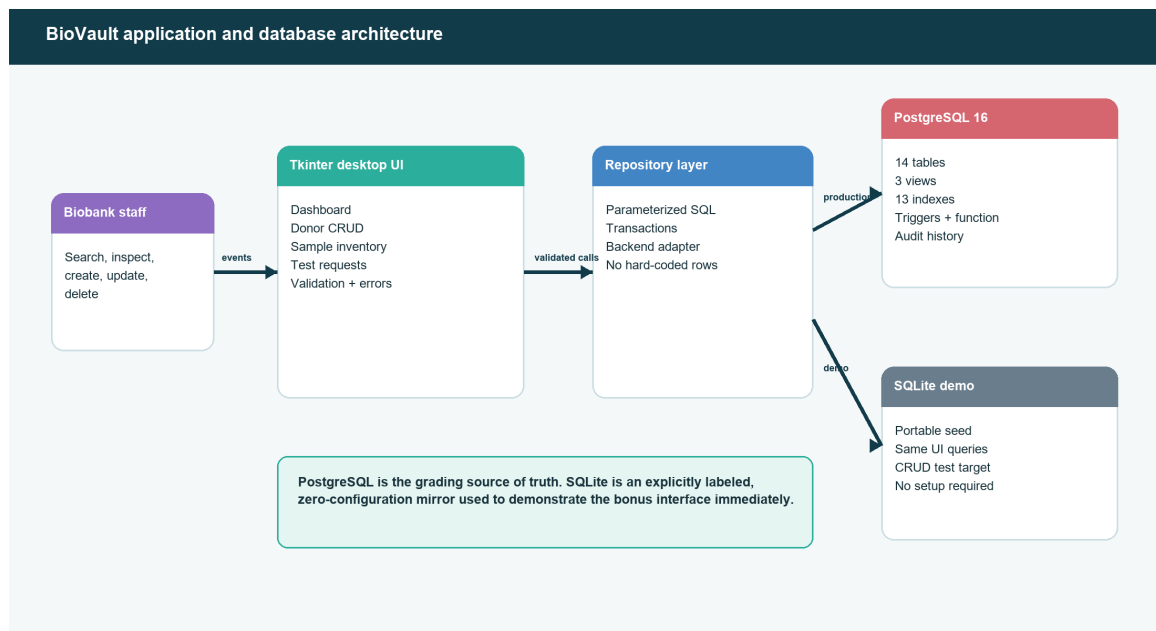


Figure 2: BioVault desktop application architecture.

The application separates:

1. `src/app.py` — Tkinter windows, forms, tables, messages;
2. `src/repository.py` — validated data operations and SQL;
3. `src/database.py` — connection and transaction management;
4. PostgreSQL or the SQLite demonstration database.

This separation makes UI changes less likely to damage SQL and makes the data layer testable without clicking windows.

31.3 Why Tkinter?

Tkinter is included with common Python installers on Windows and is suitable for a small academic desktop interface. The bonus requirement is genuine database CRUD, not a particular web framework.

The interface offers:

- live dashboard counts;
- donor, sample, and test-request views;

- cross-field search;
- validated donor create/update/delete;
- database error messages.

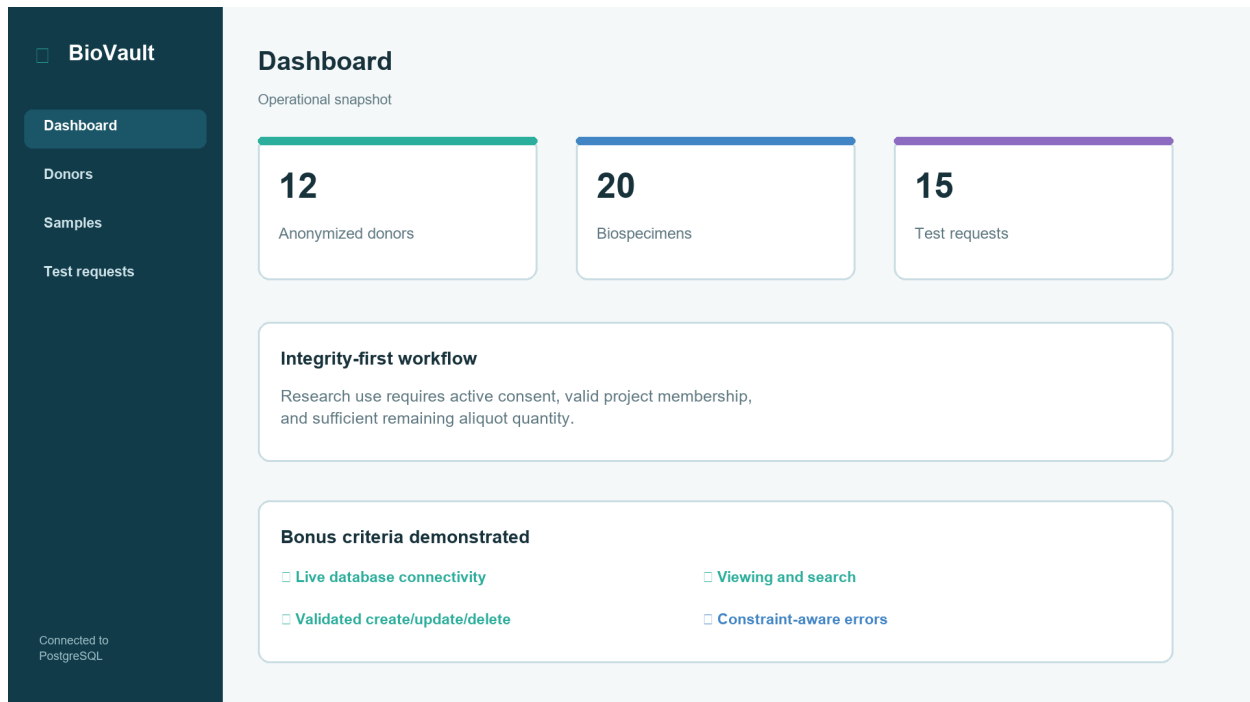


Figure 3: BioVault dashboard in the submitted application.

31.4 PostgreSQL and SQLite modes

The same UI can use:

- PostgreSQL — official implementation and source of truth;
- SQLite — zero-configuration demonstration mirror.

The adapter exposes:

```
@property
def placeholder(self) -> str:
    return "%s" if self.is_postgres else "?"
```

and:

```
def table(self, name: str) -> str:
    return f"biobank.{name}" if self.is_postgres else name
```

This small compatibility boundary avoids filling repository methods with backend-specific branches.

SQLite is not a perfect substitute:

- type enforcement differs;
- concurrency differs;
- there is no PL/pgSQL;
- trigger/function features differ;
- PostgreSQL-specific views and indexes may not translate.

Therefore database acceptance tests run against PostgreSQL.

32. Connections and transaction handling in Python

32.1 Context manager

The database adapter uses:

```
@contextmanager
def connection(self):
    connection = ...
    try:
        yield connection
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    finally:
        connection.close()
```

This guarantees:

- success commits;
- failure rolls back;
- the connection closes in either case;
- the original exception is not silently hidden.

32.2 Environment configuration

```
$env:BIOBANK_DATABASE_URL = `
    "postgresql://biobank:biobank_dev@localhost:55432/biobank"
python -m src.app
```

Environment configuration keeps deployment-specific addresses out of application source. Production secrets should use an approved secret manager or protected environment mechanism.

32.3 Driver

The application imports psycopg only in PostgreSQL mode:

```
import psycopg
connection = psycopg.connect(self.database_url)
```

SQLite uses Python's sqlite3 module.

32.4 Foreign keys in SQLite

SQLite requires explicit enforcement per connection:

```
connection.execute("PRAGMA foreign_keys = ON")
```

Without this, the demo could accept orphan rows even though the schema declares foreign keys.

32.5 Connection scope

The submitted app opens short-lived connections per repository operation. This is simple and safe for a small desktop project.

A production service with high request volume would usually use a connection pool, timeouts, observability, and retry policies. Adding a pool to this educational app would increase complexity without improving the demonstrated requirement.

33. Safe SQL from application code

33.1 Parameterized queries

Never build user values directly into SQL text:

```
# Unsafe
sql = f"SELECT * FROM donors WHERE donor_code = '{user_input}'"
```

Use parameters:

```
sql = """
    SELECT donor_id, donor_code
    FROM donors
    WHERE donor_id = %s
"""
cursor = connection.execute(sql, (donor_id,))
```

The driver sends code and values separately, preventing a value from becoming SQL syntax.

33.2 Why placeholders differ

- Psycopg uses %s.
- SQLite uses ?.

The adapter returns the correct marker. Do not add quotes around the marker:

```
WHERE donor_id = %s
```

not:

```
WHERE donor_id = '%s'
```

33.3 Dynamic identifiers are different

Parameters represent values, not table or column names. BioVault's dynamic table names come only from hard-coded internal tuples:

```
tables = ("donors", "samples", "test_requests")
```

Never accept an arbitrary user-supplied table name into an f-string. For legitimate dynamic PostgreSQL identifiers, use a strict allowlist and the driver's identifier-composition tools.

33.4 Search pattern

```
pattern = f"%{search.strip()}%"
...
WHERE LOWER(donor_code) LIKE LOWER(%s)
      OR LOWER(COALESCE(ethnicity, '')) LIKE LOWER(%s)
```

The percent signs belong to the parameter value. The value remains parameterized.

33.5 Return only required fields

Repository list methods name their columns. This supports a stable UI contract and avoids accidental data exposure.

34. Validation and error handling

34.1 Validation layers

BioVault validates donor input in Python and PostgreSQL.

Application validation:

- gives immediate, friendly messages;
- normalizes casing and whitespace;
- checks date formatting before sending SQL.

Database validation:

- protects writes from every client;
- prevents race-condition violations;
- preserves correctness if the UI contains a bug.

These layers are complementary.

34.2 Cleaning first

```
cleaned = {
    "donor_code": str(data.get("donor_code", "")).strip().upper(),
    "ethnicity": str(data.get("ethnicity", "")).strip(),
    ...
}
```

Cleaning produces a predictable representation. It should not silently invent meaning. Blank birth year becomes null because the domain permits “unknown.”

34.3 Regex validation

```
DONOR_CODE_PATTERN = re.compile(r"^[BIO-D]\d{4}$")
```

- ^ start;
- literal BIO-D;
- \d{4} exactly four digits;
- \$ end.

The database has an equivalent PostgreSQL check, so the format remains protected outside Python.

34.4 Enumerated application sets

```
VALID_DONOR_STATUSES = {
    "ACTIVE", "INACTIVE", "WITHDRAWN", "DECEASED"
}
```

The set matches the database check. When changing an allowed state, update and test every layer deliberately.

34.5 Dates

```
date.fromisoformat(cleaned["registered_on"])
```

requires ISO-style YYYY-MM-DD. An unambiguous standard avoids whether 03/04/2026 means March 4 or April 3.

34.6 Do not hide exceptions

The connection manager rolls back and re-raises. The UI can translate known errors into friendly messages. Logging and troubleshooting still need the technical cause.

Bad:

```
try:
    ...
except Exception:
    pass
```

This can make a failed write look successful.

35. Repository pattern and real CRUD

35.1 Repository responsibility

BiobankRepository provides methods such as:

```
dashboard_counts()
list_donors(search)
get_donor(donor_id)
create_donor(data)
update_donor(donor_id, data)
delete_donor(donor_id)
```

```
list_samples(search)
list_test_requests(search)
```

The UI asks for domain operations without containing SQL details.

35.2 Create and return the identifier

PostgreSQL:

```
cursor = connection.execute(
    sql + " RETURNING donor_id",
    values,
)
return int(cursor.fetchone()[0])
```

SQLite:

```
cursor = connection.execute(sql, values)
return int(cursor.lastrowid)
```

The backend-specific difference remains at a small boundary.

35.3 Detect missing updates/deletes

```
if cursor.rowcount != 1:
    raise LookupError(f"Donor {donor_id} does not exist.")
```

An UPDATE matching zero rows is syntactically successful, but the requested business operation did not happen. Checking rowcount makes the behavior explicit.

35.4 Protected deletion

Deleting a referenced donor raises a foreign-key error. The interface reports that scientific history protects the record. The correct workflow is normally to change lifecycle status.

35.5 Read queries are also real database features

The sample view joins samples, types, events, and donors. The test view joins requests, samples, test types, and researchers. Search terms go to SQL, and dashboard counts are executed against live tables. This proves the interface is not displaying hard-coded lists.

Part VIII — Testing, Evidence, Delivery, and Defense

36. Testing databases and applications

36.1 Testing pyramid for this project

BioVault uses:

- schema/constraint tests in PostgreSQL;
- business-trigger acceptance tests in PostgreSQL;
- repository and validation tests in Python;
- manual UI demonstration;
- deterministic full rebuild.

Different tests answer different questions.

36.2 Positive and negative tests

Positive test: valid usage succeeds and quantity decreases.

Negative tests:

- invalid code fails;
- impossible quantity fails;
- overdraw fails;
- expired consent fails;
- unassigned researcher fails;
- deleting referenced donor fails.

Negative tests are especially important for constraints. A rule is not proved until an invalid state is attempted.

36.3 Arrange, Act, Assert

Arrange: create a temporary database and valid donor input

Act: create, read, update, and delete the donor

Assert: each observed state matches the expected state

The Python CRUD round-trip follows this structure.

36.4 Isolated test database

The repository fixture creates a fresh SQLite file inside pytest's temporary directory:

```
database_path = tmp_path / "test_biobank.db"
database = Database(f"sqlite:/// {database_path}")
database.initialize_sqlite_demo()
```

Each test run starts from a known state and does not damage a developer's manual demo database.

36.5 Parameterized test

```
@pytest.mark.parametrize(
    ("field", "value", "message"),
    [
        ("donor_code", "PERSON-12", "BIO-D0000"),
        ("sex_at_birth", "OTHER", "sex-at-birth"),
        ("birth_year", "not-a-year", "number"),
        ("registered_on", "27/07/2026", "YYYY-MM-DD"),
    ],
)
```

One test structure checks several invalid inputs. This reduces repeated test code while keeping each case visible.

36.6 Test commands

Python:

```
$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD = "1"
python -m pytest -q
```

PostgreSQL:

```
powershell -ExecutionPolicy Bypass `
-File scripts/test_database.ps1 -KeepDatabase
```

The first command disables unrelated globally installed pytest plugins so the project test environment remains predictable.

36.7 What row counts do and do not prove

Counts prove that minimum test-data volume exists. They do not prove:

- relationships are correct;
- constraints reject bad data;
- consent logic works;
- UI writes are persistent;
- concurrency is safe.

A professional evidence set combines structural, behavioral, and interface tests.

36.8 Test evidence

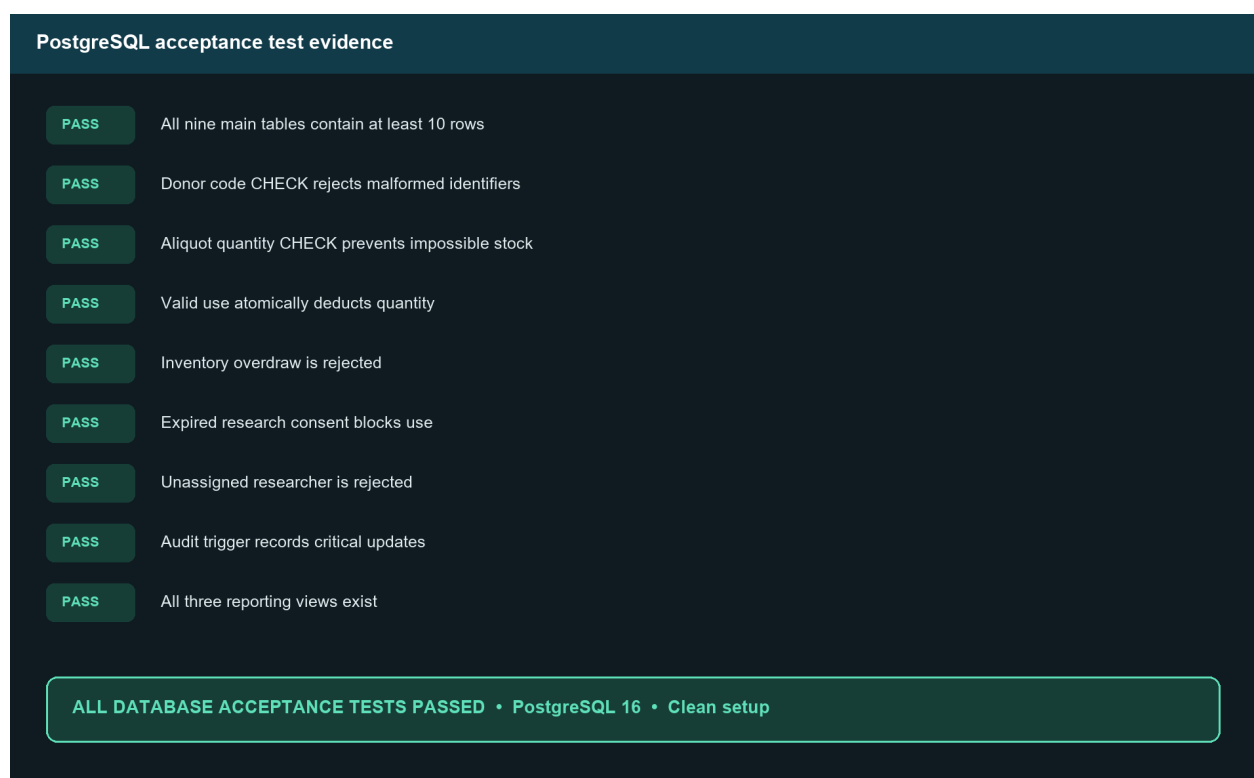


Figure 4: PostgreSQL acceptance-test evidence.

The repository includes text and image evidence, but commands remain the source of reproducible truth. A screenshot alone can be stale or edited.

37. Debugging method

37.1 Read the first real error

Later errors may be consequences. In a setup script, a missing table can produce many follow-on failures. Fix the earliest root error, rebuild cleanly, then reevaluate.

37.2 Reduce the problem

If a six-table query is wrong:

1. run the base table;
2. add one join;
3. inspect keys and row count;
4. add the next join;
5. add filters;
6. add aggregation last.

If an insert fails:

1. read the named constraint;
2. inspect the supplied row;
3. inspect referenced parent rows;
4. reproduce with the smallest statement;
5. correct data or logic, not the constraint just to silence it.

37.3 Use transactions for experiments

```
BEGIN;
-- reproduce update or trigger behavior
SELECT ...;
ROLLBACK;
```

This allows realistic behavior without permanently changing the dataset.

37.4 Inspect schema, do not guess

Useful psql commands:

```
\dn
\dt biobank.*
\d+ biobank.aliquots
\dv biobank.*
\df biobank.*
```

Useful catalog/information queries:

```
SELECT constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'biobank'
AND table_name = 'aliquots';
```

37.5 Common error categories

Error	Likely cause
relation does not exist	wrong schema/search path or setup order
foreign-key violation	missing parent or protected deletion
check violation	value contradicts row rule
unique violation	duplicate candidate key/position
not-null violation	required input missing
current transaction is aborted	earlier statement failed; rollback needed
connection refused	server/port unavailable

Error	Likely cause
password authentication failed	wrong credentials or role

38. Git and repository organization

38.1 Why version control matters

Git records meaningful snapshots of source files. It helps:

- recover earlier work;
- review changes;
- collaborate;
- connect evidence to an exact version;
- publish a reproducible submission.

Git is not a database backup strategy, and GitHub is not automatically private.

38.2 Useful workflow

```
git status
git diff
git add course scripts README.md
git commit -m "Add beginner-to-advanced database masterclass"
git push origin main
```

Review status and diff before committing. Do not commit secrets, personal data, local database files, or generated caches.

38.3 Repository structure communicates quality

The project separates:

```
sql/           database implementation
src/           bonus application
tests/         application tests
docs/          design documentation
diagrams/      ERD and architecture
evidence/      test evidence
presentation/  defense material
course/        this learning book
scripts/       repeatable automation
```

A reviewer can find the required artifact without searching through random filenames.

38.4 Generated and source artifacts

Keep both:

- editable source, such as Markdown, Mermaid, Python, and SQL;
- submission formats, such as PDF, DOCX, PNG, PPTX, and MP4.

Source makes the work maintainable. Final formats make it easy to assess.

39. Report, presentation, demo, and oral defense

39.1 Tell one coherent story

A strong defense follows the engineering chain:

```
Problem
-> requirements and rules
-> ER model
```

```
-> relational mapping and 3NF
-> constraints and database logic
-> representative queries
-> connected CRUD interface
-> tests and evidence
-> limitations and future work
```

Do not present tables as an unexplained list.

39.2 Defend decisions, not syntax alone

Weak:

I used a foreign key because the assignment asked for one.

Strong:

`samples.collection_event_id` is a non-null foreign key because every sample must have one traceable collection origin. Restrictive deletion prevents that origin from disappearing while a sample remains.

39.3 Live demonstration sequence

1. show repository structure;
2. rebuild PostgreSQL from scripts;
3. show main table counts;
4. run a lineage join;
5. run an authorized usage inside a rollback transaction;
6. show quantity deduction before rollback;
7. demonstrate a rejected overdraw or expired consent;
8. open the UI and show live counts/search;
9. create, update, and delete an unreferenced demo donor;
10. run tests.

Keep a deterministic runbook. Do not improvise destructive commands during the defense.

39.4 Likely defense questions

Why not one giant table?

It repeats independent facts and creates update, insert, and delete anomalies. The 3NF design stores each determinant once and reconnects facts through keys.

Why use both numeric IDs and readable codes?

IDs provide stable compact joins; readable codes support users. Unique constraints preserve both identities.

Why not store donor_id directly in aliquots?

The donor is functionally determined through aliquot → sample → collection event. Copying it risks contradictory lineage.

Why is project membership a table?

The relationship is M:N and owns role and date attributes.

Why is the usage trigger necessary?

The invariant spans consent, membership, project validity, aliquot state, and a concurrency-safe quantity update. A simple row check cannot express it.

Why FOR UPDATE?

It serializes competing modifications to the same aliquot so two sessions cannot both approve consumption based on stale quantity.

Why not cascade donor deletion?

It would destroy scientific lineage and usage evidence. Restrictive deletion preserves traceability.

Why SQLite if PostgreSQL is required?

Only for instant UI demonstration. PostgreSQL is the official implementation and the target of database acceptance tests.

39.5 Be honest about limitations

Examples:

- the project is an educational single-site prototype;
- direct identifiers and real clinical data are deliberately excluded;
- permissions, encryption, backups, retention, and disaster recovery need production infrastructure;
- the hierarchy check blocks only immediate self-parenting, not every possible long cycle;
- units are stored and labeled but not automatically converted;
- the demo UI exposes donor CRUD rather than every table;
- the rehearsal video requires the submitting student's own narration if the instructor requires personal voice.

Knowing limitations demonstrates understanding, not weakness.

Part IX — Guided Labs

40. Lab 1: create a tiny clean schema

Goal

Practice keys, types, constraints, and a one-to-many relationship.

Create `training_donors` and `training_samples` in a disposable schema.

Requirements:

- numeric identity primary keys;
- unique codes;
- donor code format TR-D0001;
- positive sample quantity;
- non-null donor foreign key;
- restrictive deletion;
- sample status limited to AVAILABLE, USED, DESTROYED.

Then:

1. insert two donors;
2. insert three samples;
3. attempt a duplicate code;
4. attempt a negative quantity;
5. attempt to delete a referenced donor;
6. record the error from each invalid action.

41. Lab 2: normalize a laboratory spreadsheet

Start with:

```
RequestCode, SampleCode, SampleTypeCode, SampleTypeName,  
TestCode, TestName, ResultUnit, ResearcherCode,  
ResearcherName, ResearcherEmail, RequestedOn, Status, Result
```

Tasks:

1. state the likely candidate key;
2. list functional dependencies;
3. identify update, insert, and delete anomalies;
4. decompose to 3NF;
5. identify primary and foreign keys;
6. draw cardinalities;
7. explain whether result belongs to test type or request.

42. Lab 3: query the BioVault lineage

Write one query returning:

- donor code;
- collection event code and timestamp;
- sample code and type;
- aliquot code;
- current quantity and unit;
- storage box code;
- immediate parent unit name.

Requirements:

- explicit JOIN ... ON;
- qualified columns;
- one row per aliquot;

- sort by donor, sample, aliquot;
- do not use `SELECT *`.

Explain why joining `test_requests` would change the result grain.

43. Lab 4: analytical SQL

Write queries for:

1. donors who have no collection events;
2. sample types with at least three accepted samples;
3. each project's completed-test count and average turnaround;
4. the latest usage per aliquot using `ROW_NUMBER`;
5. the cumulative quantity used per project;
6. full storage paths with a recursive CTE;
7. projects whose usage-event count is above the average project count.

For every query, write the intended grain before writing SQL.

44. Lab 5: transaction and concurrency reasoning

Assume aliquot 10 has 0.100 mL.

1. Session A begins and locks aliquot 10.
2. Session B requests the same row `FOR UPDATE`.
3. Session A deducts 0.070 and commits.
4. Session B continues and wants to deduct 0.050.

Answer:

- what does B do while A owns the lock?
- what quantity should B see afterward?
- should B succeed?
- what would happen if both applications checked quantity before beginning their write transactions?

Then use two `psql` windows to demonstrate the wait in a disposable transaction. Always finish with `ROLLBACK` if you are using submitted seed data.

45. Lab 6: add one application feature safely

Add a read-only "Available Inventory" tab to a copy of the app.

Requirements:

- repository method queries `v_available_inventory` in PostgreSQL;
- values are parameterized;
- search supports sample/aliquot/type/location;
- UI contains no SQL;
- automated test verifies a plasma search;
- failure rolls back and surfaces a useful message.

Before coding, define:

- method input and output;
- backend behavior for the SQLite demo;
- expected empty result;
- one positive and one negative test.

46. Lab 7: design acceptance tests

For each business rule, write:

1. precondition;
2. action;
3. expected database result;
4. expected persistent state after failure.

Rules:

- expired consent blocks usage;
- non-member blocks usage;
- overdraw blocks usage;
- valid usage deducts exactly once;
- critical donor update creates one audit row;
- deleting a referenced donor fails;
- duplicate storage position fails.

Part X — Answer Key and Explanations

47. Checkpoint answers

47.1 Chapter 1

1. BIO-D0007 is structured data when labeled as a donor code.
2. A microscope image is unstructured/binary content.
3. Magnification is metadata describing the image.
4. The count report is information produced from organized data.

47.2 Chapter 3

1. aliquot quantity: NUMERIC(14,3);
2. active flag: BOOLEAN;
3. exact arrival instant: TIMESTAMPTZ;
4. restriction explanation: TEXT;
5. project start: DATE.

47.3 Chapter 5

Names and other direct identifiers change the privacy classification and threat model. They require stronger authorization, governance, retention, breach response, and potentially legal controls. The research workflow needs a pseudonymous code, so storing names adds risk without supporting the stated scope.

47.4 Chapter 8

1. samples.sample_id: surrogate primary key.
2. samples.sample_code: natural/business candidate key, enforced unique.
3. samples.collection_event_id: foreign key.
4. (project_id, researcher_id): composite primary key; both columns are also foreign keys.

47.5 Chapter 13

- server: running PostgreSQL software;
- database: named logical database managed by that server;
- schema: namespace inside the database;
- table: relation inside the schema;
- psql: client that connects and sends commands.

47.6 Chapter 25

Each index consumes storage and must be maintained on writes. Indexes with low selectivity or no matching workload may never be chosen. Extra maintenance, cache pressure, and planning choices can make the total system slower.

48. Lab solution sketches

These are solution **sketches**, not the only correct answers. Compare reasoning, not whitespace.

48.1 Lab 1

```
CREATE SCHEMA training;
SET search_path TO training;

CREATE TABLE training_donors (
    donor_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    donor_code VARCHAR(8) NOT NULL UNIQUE,
    CONSTRAINT ck_training_donor_code
```



```

        CHECK (donor_code ~ '^TR-D[0-9]{4}$')
    );

CREATE TABLE training_samples (
    sample_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    sample_code VARCHAR(12) NOT NULL UNIQUE,
    donor_id BIGINT NOT NULL,
    quantity NUMERIC(10,3) NOT NULL CHECK (quantity > 0),
    sample_status VARCHAR(12) NOT NULL
        CHECK (sample_status IN ('AVAILABLE', 'USED', 'DESTROYED')),
    CONSTRAINT fk_training_sample_donor
        FOREIGN KEY (donor_id)
        REFERENCES training_donors(donor_id)
        ON DELETE RESTRICT
);

```

The invalid statements should be attempted individually so one aborted transaction does not hide the next result.

48.2 Lab 2

Likely dependencies:

```

request_code -> sample_code, test_code, researcher_code,
               requested_on, status, result
sample_code -> sample_type_code
sample_type_code -> sample_type_name
test_code -> test_name, result_unit
researcher_code -> researcher_name, researcher_email

```

Relations:

```

SAMPLE_TYPES(sample_type_id, code, name)
SAMPLES(sample_id, code, sample_type_id)
TEST_TYPES(test_type_id, code, name, result_unit)
RESEARCHERS(researcher_id, code, name, email)
TEST_REQUESTS(request_id, code, sample_id, test_type_id,
               researcher_id, requested_on, status, result)

```

The result belongs to a request because it is the outcome of one test performed on one sample. `result_unit` belongs to test type if the catalog guarantees one unit for that type.

48.3 Lab 3

```

SELECT
    d.donor_code,
    ce.event_code,
    ce.collected_at,
    s.sample_code,
    st.type_name,
    a.aliquot_code,
    a.current_quantity,
    a.quantity_unit,
    box.location_code AS box_code,
    parent.unit_name AS parent_unit
FROM donors AS d
JOIN collection_events AS ce ON ce.donor_id = d.donor_id
JOIN samples AS s ON s.collection_event_id = ce.collection_event_id
JOIN sample_types AS st ON st.sample_type_id = s.sample_type_id
JOIN aliquots AS a ON a.sample_id = s.sample_id
JOIN storage_units AS box ON box.storage_unit_id = a.storage_unit_id
LEFT JOIN storage_units AS parent
    ON parent.storage_unit_id = box.parent_storage_unit_id
ORDER BY d.donor_code, s.sample_code, a.aliquot_code;

```

Joining test requests directly can create one row per aliquot–request combination, not one row per aliquot.

48.4 Lab 4 selected solutions

Donors with no events:

```
SELECT d.donor_code
FROM donors AS d
WHERE NOT EXISTS (
    SELECT 1
    FROM collection_events AS ce
    WHERE ce.donor_id = d.donor_id
);
```

Types with at least three accepted samples:

```
SELECT st.type_code, st.type_name, COUNT(*) AS accepted_samples
FROM sample_types AS st
JOIN samples AS s ON s.sample_type_id = st.sample_type_id
WHERE s.quality_status = 'ACCEPTED'
GROUP BY st.sample_type_id, st.type_code, st.type_name
HAVING COUNT(*) >= 3;
```

Latest use per aliquot:

```
WITH ranked_usage AS (
    SELECT
        u.*,
        ROW_NUMBER() OVER (
            PARTITION BY aliquot_id
            ORDER BY used_on DESC, usage_id DESC
        ) AS row_num
    FROM sample_usage AS u
)
SELECT *
FROM ranked_usage
WHERE row_num = 1;
```

Above-average project usage:

```
WITH project_counts AS (
    SELECT
        p.project_id,
        p.project_code,
        COUNT(u.usage_id) AS usage_count
    FROM research_projects AS p
    LEFT JOIN sample_usage AS u ON u.project_id = p.project_id
    GROUP BY p.project_id, p.project_code
)
SELECT project_code, usage_count
FROM project_counts
WHERE usage_count > (
    SELECT AVG(usage_count) FROM project_counts
);
```

48.5 Lab 5

Session B waits while A owns the conflicting row lock. After A commits, B continues and evaluates the current committed row with 0.030 mL. Its requested 0.050 mL exceeds availability, so it must fail. If both applications checked outside the protected write transaction, both could approve based on stale 0.100 mL.

48.6 Lab 6 design

A good repository contract is:

```
def list_available_inventory(self, search: str = "") -> list[dict[str, Any]]:
    ...
```

PostgreSQL can query the view. SQLite needs an equivalent read query because the demo schema may not contain the PostgreSQL view. Use a hard-coded backend branch for the query structure, parameterize the search value in both, and keep UI code limited to calling the method and rendering returned dictionaries.

48.7 Lab 7 example

Expired consent:

Precondition: an aliquot traces to a donor whose RESEARCH_USE consent expired.

Action: call `record_sample_usage` with a use date after expiry.

Expected result: exception mentioning missing active consent.

Persistent state: no `sample_usage` row and unchanged `current_quantity`.

The final state is crucial. Merely seeing an error does not prove that an earlier deduction rolled back.

Part XI — Cheat Sheets

49. SQL statement cheat sheet

Create

```
CREATE TABLE table_name (  
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    code VARCHAR(20) NOT NULL UNIQUE,  
    parent_id BIGINT REFERENCES parent_table(id),  
    quantity NUMERIC(14,3) NOT NULL CHECK (quantity > 0)  
);
```

Insert

```
INSERT INTO table_name (code, quantity)  
VALUES ('CODE-001', 1.250)  
RETURNING id;
```

Read

```
SELECT t.code, p.name  
FROM table_name AS t  
JOIN parent_table AS p ON p.id = t.parent_id  
WHERE t.quantity > 0  
ORDER BY t.code;
```

Update

```
UPDATE table_name  
SET quantity = quantity - 0.250  
WHERE id = 1  
RETURNING id, quantity;
```

Delete

```
DELETE FROM table_name  
WHERE id = 1  
RETURNING id;
```

Aggregate

```
SELECT parent_id, COUNT(*) AS items, SUM(quantity) AS total  
FROM table_name  
GROUP BY parent_id  
HAVING COUNT(*) > 1;
```

Transaction

```
BEGIN;  
-- related operations  
COMMIT;  
-- or ROLLBACK;
```

CTE

```
WITH summary AS (
    SELECT parent_id, COUNT(*) AS item_count
    FROM table_name
    GROUP BY parent_id
)
SELECT *
FROM summary
WHERE item_count > 2;
```

Window

```
SELECT
    code,
    quantity,
    DENSE_RANK() OVER (ORDER BY quantity DESC) AS quantity_rank
FROM table_name;
```

50. Design review cheat sheet

For every table:

- What does one row mean?
- What is the primary key?
- What other candidate keys exist?
- Which values are required?
- Which values use controlled domains?
- Which foreign keys express relationships?
- What should happen on parent deletion?
- Is every non-key attribute about this row's key?
- Which indexes match real access paths?
- Who inserts, updates, reads, and deletes rows?
- Which tests prove the rules?

For every query:

- What is the intended output grain?
- Which table establishes that grain?
- Can a join multiply rows?
- Should missing matches remain?
- Are nulls handled intentionally?
- Are units compatible before aggregation?
- Is order explicitly specified?
- Are application values parameterized?

For every write workflow:

- What is the complete atomic unit?
- Which constraints can express the rule?
- Which multi-row checks are needed?
- Can concurrent sessions violate the invariant?
- Which rows must be locked?
- What should remain after failure?
- Is the change auditable?

51. PostgreSQL command cheat sheet

```
# Start PostgreSQL
docker compose up -d database
```

```
# Clean setup
docker compose exec -T database \
  psql -U biobank -d biobank -f /project/sql/setup.sql

# Interactive psql
docker compose exec database psql -U biobank -d biobank

# Database acceptance tests
docker compose exec -T database \
  psql -U biobank -d biobank -f /project/sql/test_constraints.sql

# Python tests
$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD = "1"
python -m pytest -q

# Start UI in SQLite demo mode
python -m src.app

# Start UI against PostgreSQL
$env:BIOBANK_DATABASE_URL = `
  "postgresql://biobank:biobank_dev@localhost:55432/biobank"
python -m src.app

# Stop PostgreSQL and keep data
docker compose down
```

Inside psql:

```
\conninfo      show current connection
\dn            list schemas
\d+ biobank.*  list tables
\d+ biobank.aliquots describe a table
\dv biobank.*  list views
\df biobank.*  list functions
\q            quit
```

Part XII — Glossary

52. Database glossary

ACID: atomicity, consistency, isolation, and durability; core transaction properties.

Aggregate: a calculation over several rows, such as count or sum.

Aliquot: a controlled portion prepared from a parent biospecimen.

Anomaly: an unwanted insertion, update, or deletion problem caused by poor data organization.

Attribute: a property of an entity; usually implemented as a column.

Audit log: history describing important changes.

Biospecimen: biological material collected for testing or research.

Business rule: a precise statement governing valid operations or data.

Candidate key: a minimal set of attributes that uniquely identifies a row.

Cardinality: maximum relationship participation, such as one-to-many.

Check constraint: row-level Boolean rule enforced by the DBMS.

Client: program that connects to a database server and sends commands.

Column: named, typed attribute in a table.

Commit: permanently accept a transaction's changes.

Composite key: key containing more than one column.

Concurrency: overlapping work by multiple sessions.

Constraint: declarative rule limiting valid database states.

Correlated subquery: subquery that references a row from its outer query.

CRUD: create, read, update, delete.

CTE: Common Table Expression; a named query block introduced by WITH.

Data: recorded facts or observations.

Database: organized collection of data.

DBMS: software that manages databases.

DDL: SQL commands defining structures.

DML: SQL commands inserting or changing rows.

Entity: distinguishable thing, concept, or event to be stored.

ERD: Entity–Relationship Diagram.

Foreign key: columns whose values must match a referenced candidate key.

Functional dependency: rule $X \rightarrow Y$ meaning X determines Y.

Grain: what exactly one row in a table or result represents.

Index: auxiliary structure that can speed row lookup.

Inner join: returns matching row combinations.

Isolation: rules governing transaction visibility and interference.

Join: relational operation combining corresponding rows.

JSONB: PostgreSQL binary JSON type supporting structured operations.

Left join: keeps every left row and adds matching right data when available.

Lock: mechanism coordinating conflicting concurrent operations.

Metadata: data that describes other data or materials.

Normalization: structured decomposition based on keys and dependencies.

NULL: missing, unknown, or inapplicable value.

Participation: minimum relationship involvement, optional or mandatory.

Partial index: index containing only rows satisfying a predicate.

Parameterized query: SQL sent separately from user values.

Primary key: chosen candidate key used as the main row identity.

Pseudonymization: replacing direct identity with a controlled code while a separate linking capability may still exist.

Recursive CTE: CTE that repeatedly references its accumulated result.

Referential integrity: guarantee that foreign-key references remain valid.

Relation: mathematical foundation of a relational table.

Repository pattern: application layer that groups persistent data operations.

Rollback: discard a transaction's uncommitted changes.

Row: one tuple/record in a relation.

Schema: logical structure, or a PostgreSQL namespace of objects.

SQL: Structured Query Language.

Subquery: query nested inside another SQL statement.

Surrogate key: system-introduced identifier without domain meaning.

Table: SQL representation of a relation.

Test fixture: controlled starting environment or data for a test.

Third Normal Form (3NF): normalized form excluding partial and transitive dependencies of non-key attributes.

Transaction: atomic unit of database work.

Trigger: function automatically invoked by a table event.

View: named stored query definition.

Window function: calculation across related rows that preserves individual result rows.

53. Final learning path: from student to expert

Stage 1 — Explain

Without looking at notes, explain:

- database versus DBMS;
- table, row, column, type, and null;
- primary, candidate, composite, and foreign keys;
- one-to-many versus many-to-many.

If you cannot explain a word simply, revisit the chapter.

Stage 2 — Model

Take a new small domain, such as a cell-culture laboratory. Write scope, ten business rules, an entity list, and an ERD. Walk three scenarios through it.

Stage 3 — Normalize

Start from one deliberately messy sheet. Write functional dependencies and decompose through 1NF, 2NF, and 3NF. Explain which anomaly each decomposition removes.

Stage 4 — Build and query

Create the schema from scratch, seed boundary states, and write:

- filters;
- multi-table joins;
- aggregation and HAVING;
- subqueries and EXISTS;
- CTEs;
- one recursive query;
- window functions;
- reusable views.

Stage 5 — Protect

Add constraints, transactions, a concurrency-sensitive workflow, auditing, and negative tests. Reproduce one race in two sessions and show how locking fixes it.

Stage 6 — Integrate

Build a small interface with:

- parameterized reads;
- complete CRUD for one entity;
- validation in application and database;
- explicit transaction handling;
- automated repository tests.

Stage 7 — Defend

For every decision, answer:

What problem does this solve, what alternative existed, what trade-off did we accept, and what evidence proves it works?

That question marks the change from memorizing syntax to thinking like a database engineer.

54. Final project self-assessment

Score each statement from 0 to 2:

- 0 — I cannot explain it yet.
- 1 — I recognize it but need notes.
- 2 — I can explain and demonstrate it.

Skill	Score
I can define the project scope and stakeholders.	
I can explain every entity and row grain.	
I can identify every key and relationship.	
I can defend the 3NF decomposition.	
I can explain each major constraint.	
I can write joins without guessing.	
I can use grouping, subqueries, CTEs, and windows.	
I can explain every BioVault view and index.	
I can demonstrate ACID and rollback.	
I can explain why the row lock prevents overdraft.	
I can trace the usage trigger line by line.	
I can explain the audit design and limitations.	
I can trace UI → repository → connection → database.	
I can explain parameterization and SQL injection.	
I can run and interpret all tests.	
I can perform the live demo without hidden steps.	
I can state honest limitations and future work.	

Maximum: 34.

- 0–12: repeat Parts I–III and perform Labs 1–2.
- 13–22: practice Parts IV–V and Labs 3–4.
- 23–29: focus on Parts VI–VIII and Labs 5–7.
- 30–34: you are ready to defend; now practice unscripted changes.

55. References and further study

The course explanations are grounded in the submitted BioVault implementation and the following authoritative resources:

1. National Cancer Institute, *NCI Best Practices for Biospecimen Resources*, 2026. <https://biospecimens.cancer.gov/resources/sops/docs/2026-NCI-Best-Practices.pdf>
2. National Cancer Institute, *Biorepositories and Biospecimen Research Branch overview*. <https://biospecimens.cancer.gov/about/overview.asp>
3. International Organization for Standardization, *ISO 20387:2018 — Biotechnology — Biobanking — General requirements for biobanking*. <https://www.iso.org/standard/67888.html>
4. PostgreSQL Global Development Group, *PostgreSQL Documentation*. <https://www.postgresql.org/docs/current/>
5. Python Software Foundation, *Graphical User Interfaces with Tk*. <https://docs.python.org/3/library/tk.html>
6. Python Software Foundation, *sqlite3 — DB-API 2.0 interface for SQLite*. <https://docs.python.org/3/library/sqlite3.html>
7. Psycopg Project, *Psycopg 3 documentation*. <https://www.psycopg.org/psycopg3/docs/>
8. Docker, *Docker Compose documentation*. <https://docs.docker.com/compose/>
9. Git Project, *Git documentation*. <https://git-scm.com/doc>

Repository sources to study beside this book

- docs/business_rules.md
- docs/normalization.md
- docs/relational_schema.md
- docs/data_dictionary.md
- sql/create_tables.sql
- sql/triggers_procedures.sql
- sql/views.sql
- sql/queries.sql
- sql/test_constraints.sql
- src/database.py
- src/repository.py
- tests/test_repository.py
- presentation/defense_questions.md

Closing note

Database skill is not measured by the number of SQL keywords you remember. It is measured by whether you can preserve meaning and correctness as requirements, data volume, users, and failure modes become more complicated.

Start with the fact. State the rule. Model the relationship. Enforce the invariant. Test the failure. Then make the interface pleasant.

That is the path from absolute zero to confident database engineering.

56. Table-by-table BioVault deep dive

This chapter is a final guided tour of the physical model. Use it beside `sql/create_tables.sql`. For each table, pay attention to the **grain**—the meaning of one row—because grain controls keys, joins, and aggregation.

56.1 donors

Grain: one pseudonymous research donor.

`donor_id` is the internal identity and `donor_code` is the human-readable candidate key. Demographic values are deliberately limited. The table does not store name, address, phone, national identifier, or a re-identification key.

The lifecycle status describes whether the donor record is active, inactive, withdrawn, or associated with a deceased donor. Status does not physically delete history. `registered_on` is a business date; `created_at` and `updated_at` are technical timestamps.

Think carefully about the difference between withdrawal and deletion. Withdrawal can affect what future use is authorized while existing scientific and regulatory history may need to remain.

56.2 consent_types

Grain: one controlled category of permission.

This is a lookup relation. `consent_code` is a stable machine-friendly key such as `RESEARCH_USE`; `consent_name` is a readable label. Separating the catalog prevents every consent record from independently spelling and describing its purpose.

Do not confuse a consent **type** with a signed consent record. A type is a definition; a row in `consents` is a donor-specific version with dates and status.

56.3 consents

Grain: one version of one donor's consent for one purpose.

The unique tuple (`donor_id`, `consent_type_id`, `version_no`) prevents the same document version from being recorded twice. Several versions remain possible. This matters because permission changes over time and historical decisions must be evaluated using the usage date, not only today's status.

The date check prevents expiry before grant. The usage trigger also evaluates:

```
granted_on <= used_on
and (expires_on is null or expires_on >= used_on)
and consent_status = ACTIVE
```

That is a temporal rule. A simple foreign key to a consent row would prove that some consent exists, but not that it authorized this purpose on this date.

56.4 researchers

Grain: one authorized research or laboratory staff member.

Both researcher code and email are candidate keys. `is_active` describes the general account/person status. Project-specific participation belongs in `project_researchers`, because the same person can have different roles and dates in different projects.

A production identity system might keep authentication credentials elsewhere. This table stores the domain profile, not passwords.

56.5 research_projects

Grain: one ethics-approved research project.

Project code and ethics approval code are separately unique. The lead researcher is a required one-to-many reference, while broader membership is M:N. Start date, optional end date, and status define the administrative authorization interval used by the sample-usage trigger.

The schema permits a lead researcher reference even before the bridge row is added. A stronger workflow rule could require the lead to also appear as an active project member. That cross-table rule would require additional database logic and tests.

56.6 project_researchers

Grain: one researcher's membership in one project.

The composite primary key prevents duplicate membership. `project_role`, `joined_on`, and `left_on` describe the pair. A nullable `left_on` means the membership has no recorded end.

The trigger checks membership dates against `sample_usage.used_on`. It does not accept a person merely because they were ever a member.

ON DELETE CASCADE applies from project to membership because an association cannot exist without its project. The researcher side remains restrictive to avoid silently removing a person's historical assignments.

56.7 sample_types

Grain: one controlled biospecimen classification.

The type owns its name, default unit, and expected temperature range. Samples reference it rather than copying these facts. This is a direct 3NF decision.

The temperature range is metadata, not active enforcement that every selected storage box has a matching temperature. A production enhancement could validate sample/aliquot type against the storage hierarchy's effective temperature. That rule is more complex because the aliquot references a box while the temperature may be declared on an ancestor freezer.

56.8 collection_events

Grain: one specimen-collection event for one donor.

The row identifies the donor, collector, exact collection instant, site, protocol, fasting status, and notes. A collection event can produce several samples, such as plasma and serum derived from one blood collection.

This table preserves the natural lineage boundary. Storing collection attributes directly in every sample would repeat event facts and allow samples from the same event to contradict one another.

56.9 samples

Grain: one parent biospecimen received by the biobank.

A sample references one collection event and one sample type. It records parent quantity, unit, quality decision, lifecycle status, processing method, and technical timestamps.

Quality and lifecycle are different dimensions:

- `quality_status` answers whether material passed assessment;
- `sample_status` answers whether it is operationally available, reserved, depleted, or destroyed.

Combining both into one status would create a growing set of mixed values such as `ACCEPTED_AVAILABLE`, `ACCEPTED_RESERVED`, and `QUARANTINED_AVAILABLE`.

56.10 storage_units

Grain: one physical storage node.

The self-reference models facilities, rooms, freezers, racks, and boxes with one table. `location_code` is a globally unique operational locator. Capacity is optional because it is meaningful for some node types but not necessarily all.

The table check prevents an immediate self-parent but does not enforce that a box must be under a rack or prevent a multi-node cycle. Those are documented extensions. The recursive query demonstrates how a flexible hierarchy can be read.

56.11 aliquots

Grain: one independently stored and consumable portion of a sample.

The table protects two identities:

- `aliquot_code` is globally unique;
- `(storage_unit_id, position_code)` is unique within physical storage.

Initial and current quantity are stored because current inventory changes while original prepared quantity remains useful for traceability. A check keeps current quantity in the closed interval from zero to initial quantity.

The status and current quantity must remain logically aligned. The usage trigger sets status to EXHAUSTED when current quantity reaches zero. Other workflows that directly change quantity should follow the same controlled path or gain a more general invariant trigger.

56.12 test_types

Grain: one controlled laboratory test definition.

The catalog owns a readable name and optional result unit. A test with textual results may not have a unit. Keeping catalog facts here avoids repeating them on every request.

In a richer laboratory-information system, a test definition could have versioned methods, reference ranges, instruments, specimen compatibility, and structured result schemas. Those are intentionally outside this project.

56.13 test_requests

Grain: one request to perform one test on one sample.

The row connects sample, test type, requester, and optional project. It tracks request and completion dates, workflow status, and either numeric or text result.

The completion constraint guarantees a completed request has a completion date, and the date cannot precede request. It does not require a result because some completed laboratory outcomes may be textual or legitimately reported through another workflow. A stronger domain could add a rule requiring exactly one of numeric or text result based on test definition.

56.14 sample_usage

Grain: one authorized event consuming a quantity from one aliquot for one project, recorded by one researcher.

Rows should be treated as immutable scientific history. The public `record_sample_usage` function and its trigger are the intended write path.

The table stores the event facts, while the trigger validates external context. Do not store a calculated “quantity remaining after use” here as the authoritative inventory; it would duplicate `aliquots.current_quantity`. Historical stock can be reconstructed from initial quantity and ordered usage when needed, subject to any other inventory adjustments.

56.15 audit_log

Grain: one audited update or delete on a protected operational row.

The table is append-oriented. It records logical identity and JSON snapshots. Indexes support finding history for a table/record pair in reverse time order.

The audit log does not enforce business correctness. It helps investigators reconstruct changes after correctness controls and permissions have done their work.

56.16 Trace one question across the schema

Question:

Can researcher 1 use 0.050 mL from aliquot 1 for project 1 on 2026-06-15?

The database must traverse or check:

1. `aliquots` — existence, status, current quantity, and row lock;
2. `samples` — the aliquot's parent;

3. `collection_events` — donor lineage;
4. `research_projects` — status and authorized dates;
5. `project_researchers` — dated membership;
6. `consents` plus `consent_types` — active research permission;
7. `sample_usage` — new immutable event;
8. `aliquots` again — atomic deduction.

This explains why the workflow is more than a single CHECK constraint and why the data model must be correct before writing procedural logic.

57. Beginner mistakes and expert corrections

57.1 Designing from screens

Mistake: create one table for each UI form and copy whatever the screen shows.

Correction: model stable domain facts and relationships first. Screens can combine several relations through queries and views.

57.2 Adding IDs but ignoring candidate keys

Mistake: assume a generated ID prevents duplicate donors or samples.

Correction: preserve UNIQUE constraints on real business identifiers.

57.3 Treating null, zero, and blank as identical

Mistake: use 0 for unknown birth year or ' ' for no expiry date.

Correction: use null for missing/inapplicable facts and document its meaning. Use zero only when zero is a real measured value.

57.4 Storing lists in one column

Mistake: store project members as comma-separated names.

Correction: use an associative relation with foreign keys and relationship attributes.

57.5 Copying descriptive data into transaction tables

Mistake: put project title, researcher email, and donor blood type on every usage row.

Correction: store references and join current master facts. Copy a snapshot only when historical semantics explicitly require it.

57.6 Using a join without knowing grain

Mistake: join two child collections and trust the total.

Correction: state “one row per ____” before writing SQL, inspect cardinality, and pre-aggregate when required.

57.7 Confusing WHERE and HAVING

Mistake: put `COUNT(*) > 3` in WHERE.

Correction: filter source rows with WHERE; filter grouped results with HAVING.

57.8 Hiding duplicates with DISTINCT

Mistake: add DISTINCT until the output “looks right.”

Correction: inspect join keys and multiplicity. Use DISTINCT only when unique output values match the question.

57.9 Building SQL with user strings

Mistake: concatenate search text into SQL.

Correction: parameterize values, allowlist any dynamic identifiers, and keep database privileges limited.

57.10 Checking stock only in the UI

Mistake: disable a button when displayed stock is low and assume overdraw is impossible.

Correction: lock, re-read, validate, record, and deduct inside one database transaction.

57.11 Catching every error and continuing

Mistake: display “saved” after swallowing a database exception.

Correction: rollback, preserve the technical error for diagnosis, translate known failures for the user, and never claim success without evidence.

57.12 Optimizing before measuring

Mistake: add dozens of indexes or denormalize because joins “might be slow.”

Correction: start with a correct model, identify real queries, inspect EXPLAIN (ANALYZE, BUFFERS) at realistic scale, and optimize the measured bottleneck.

57.13 Testing only success

Mistake: insert one valid row and call the database tested.

Correction: attempt every important invalid state and verify both the error and the unchanged persistent state.

57.14 Memorizing the presentation

Mistake: recite words without being able to change a query.

Correction: practice deriving answers from business rules, draw relationships by hand, and make controlled live modifications in rollback transactions.

Expert behavior is not error-free behavior. It is the habit of making assumptions visible, keeping invariants close to the data, reducing problems to testable units, and using evidence to correct mistakes.